

ORIGINAL RESEARCH

Resource allocation scheduling scheme for task migration and offloading in 6G Cybertwin internet of vehicles based on DRL

Rui Wei^{1,2}  | Tuanfa Qin^{1,2}  | Jinbao Huang^{1,2}  | Ying Yang^{1,2} | Junyu Ren^{1,2}  | Lei Yang³

¹School of Computer and Electronic Information, Guangxi University, Xixiangtang, Nanning, Guangxi, People's Republic of China

²Guangxi Key Laboratory of Multimedia Communications and Network Technology, Guangxi University, Xixiangtang, Nanning, Guangxi, People's Republic of China

³Industrial Internet Research Center, Guangxi Academy of Sciences, Xixiangtang, Nanning, Guangxi, People's Republic of China

Correspondence

Tuanfa Qin, School of Computer and Electronic Information, Guangxi University, Xixiangtang, Nanning, Guangxi 530004, People's Republic of China.

Email: tfqin@gxu.edu.cn

Funding information

Innovation-driven Development project of Guangxi, China, Grant/Award Number: AA20302002-2; Key Research and Development Plan Project of Guangxi, China, Grant/Award Number: AB23026037

Abstract

As vehicular technology advances, intelligent vehicles generate numerous computation-intensive tasks, challenging the computational resources of both the vehicles and the Internet of Vehicles (IoV). Traditional IoV struggles with fixed network structures and limited scalability, unable to meet the growing computational demands and next-generation mobile communication technologies. In congested areas, near-end Mobile Edge Computing (MEC) resources are often overtaxed, while far-end MEC servers are underused, resulting in poor service quality. A novel network framework utilizing sixth-generation mobile communication (6G) and digital twin technologies, combined with task migration, promises to alleviate these inefficiencies. To address these challenges, a task migration and re-offloading model based on task attribute classification is introduced, employing a hybrid deep reinforcement learning (DRL) algorithm—Dueling Double Q Network DDPG (QDPG). This algorithm merges the strengths of the Deep Deterministic Policy Gradient (DDPG) and the Dueling Double Deep Q-Network (D3QN), effectively handling continuous and discrete action domains to optimize task migration and re-offloading in IoV. The inclusion of the Mini Batch K-Means algorithm enhances learning efficiency and optimization in the DRL algorithm. Experimental results show that QDPG significantly boosts task efficiency and computational performance, providing a robust solution for resource allocation in IoV.

1 | INTRODUCTION

The rapid development of the Internet of Things era has propelled the evolution of the Vehicle Ad Hoc Network (VANET) into the Internet of Vehicles (IoV) [1, 2]. With an increasing number of vehicle users and the advancement of technologies such as smart driving and digital driving, there comes a surge in computational service demands within IoVs. Traditional 5G technology, when faced with such immense digital task computational demands, quickly becomes inadequate [3], especially since IoV is dynamic, with congested areas often requiring more substantial computing power support. Every segment of urban roads could potentially face a service quality crisis under high user numbers. In general, IoV currently suffers from a mismatch between its computational architecture and the growing computational power demands of digital tasks [4]. Fortunately,

the emergence of 6G networks is expected to offer data transmission rates of up to 1TB/s, injecting new energy into the IoV network's computational power [5], enhancing efficiency and data exchange between vehicular entities and infrastructure. Meanwhile, the advent of digital twin Cybertwin technology has brought about a powerful transformation to the network architecture of IoVs. By decoupling interfaces between vehicular nodes and other network nodes, Cybertwin allows digital vehicle users within IoVs to fully interact with computational service nodes, ensuring vehicles receive mobile service support throughout their journey. Moreover, Cybertwin technology provides the possibility of combining further network services in IoVs with artificial intelligence technologies. Cybertwin is an advanced digital technology used to create virtual replicas of physical entities or systems. This technology collects real-time data from the entity or system and uses it to build a dynamic

This is an open access article under the terms of the [Creative Commons Attribution-NonCommercial-NoDerivs](https://creativecommons.org/licenses/by-nc-nd/4.0/) License, which permits use and distribution in any medium, provided the original work is properly cited, the use is non-commercial and no modifications or adaptations are made.

© 2024 The Author(s). *IET Communications* published by John Wiley & Sons Ltd on behalf of The Institution of Engineering and Technology.

simulation model in the digital space. This digital model can be monitored, analyzed, predicted, and optimized for the performance and state of its physical counterpart in a digital virtual platform using powerful artificial intelligence algorithms, thus achieving real-time data synchronization, advanced modeling analysis, prediction, and optimization, and ultimately full utilization and scheduling of computational resources. With the support of 6G combined with Cybertwin technology, we need to expand the computational resources within IoVs and fully utilize them.

To address this issue, we further integrate cloud platforms and edge computing technologies within IoVs. Currently, both are the primary research means to expand the computational resource structure of IoVs, but each has its limitations. Cloud platforms offer storage and computational capabilities, but due to large data volumes, limited computational power, and their distance from mobile vehicle users, they struggle to provide real-time service support in dynamic IoVs. On the other hand, deploying edge computing servers at Road Side Units (RSUs) using Mobile Edge Computing (MEC) technology is an effective solution. Such edge servers are closer to vehicle user entities and have local data storage and stronger computational capabilities. However, their stationary nature may challenge the provision of stable services for high-speed vehicles. Therefore, the combined use of both is an inevitable trend and the approach adopted by this study. Currently, optimizing the utilization of cloud and edge servers for task offloading in IoV is a key research area. Many studies have proposed intelligent offloading strategies for tasks that require high computational support, low latency, and efficiency [6–9]. 6G, Cybertwin, and MEC technologies are expected to provide sufficient computing resources for IoVs. But the mobility of vehicles and the dynamic nature of IoV are not to be overlooked. We have observed this, and a typical scenario (as shown in Figure 1) is a high-density vehicle urban street or accident scene, leading to a surge in demand for nearby MEC servers, thereby overloading the most capable MEC servers, while cloud servers have limited computational capabilities, resulting in extremely poor mobile service quality for digital vehicle users. This is a problem that IoVs urgently needs to solve. We note that it is precisely because digital vehicle users compete for computational resources of proximal MEC servers that servers away from congested sections are often idle at this moment. Our research considers migrating tasks to MEC servers with sufficient computational capacity before offloading, which may be a solution to the problem. This could effectively balance the computational resources within IoVs, achieving resource scheduling [10].

The anticipated 6G technology, operating in the THz bands, brings lower cost losses for task migration through denser networks and extremely low latency. Furthermore, integrating Network-in-box (NIB) technology from 6G into digital vehicles can enhance the vehicles' computational capabilities and data storage [11], improving user experience within the IoVs framework. These technologies also provide the basic ideas for the models in this study. To solve the aforementioned task migration problem, we model RSU and MEC as two distinct entities in Figure 2. After a vehicle issues a task offloading request,

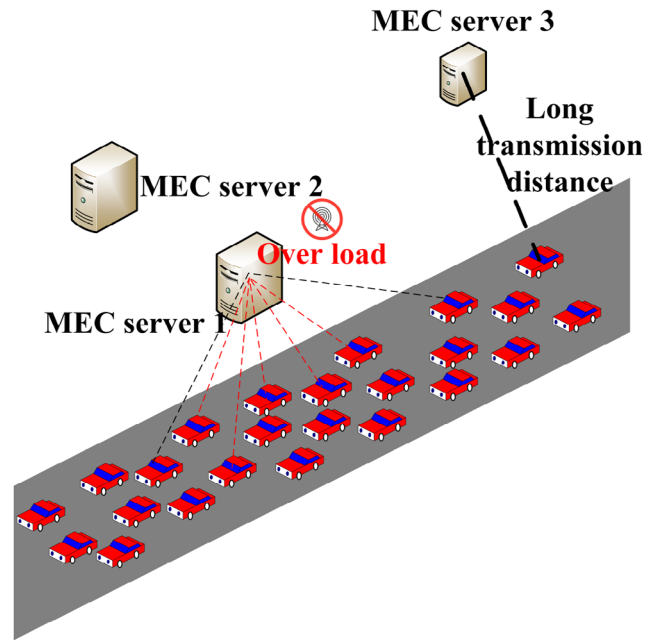


FIGURE 1 Overcrowded section.

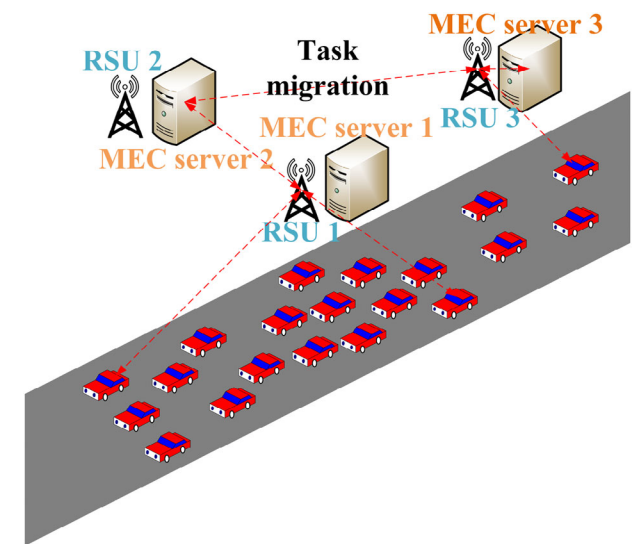


FIGURE 2 Task migration.

RSU_i is assigned to receive the task and decide whether its corresponding MEC_i should offload the task or delegate it to MEC_j [12]. Thus, the main challenges are choosing the suitable RSU and determining the optimal task offloading MEC server. Essentially, the crux of the problem lies in minimizing delay and energy consumption while optimizing load balance within MEC servers, and improving the resource utilization efficiency of MEC servers within IoVs, constituting a constrained multi-objective optimization problem.

How to ensure full computational resource support for digital tasks in IoVs and find optimal solutions for migration and offloading challenges are the focal points of this study. We recognize that machine learning techniques such as deep

learning and multi-agent reinforcement learning have proven effective in solving complex problems within the Internet of Things domain [13, 14]. Simultaneously, the Cybertwin layer based on digital twinning technology can seamlessly integrate with artificial intelligence algorithms, facilitating the deployment of Deep Reinforcement Learning (DRL) algorithms to circumvent the complexity of directly deploying traditional heuristic and AI algorithms in IoVs. Specifically, Cybertwin technology enables virtual experiments and system optimization, meaning that the Cybertwin layer can collect state information from RSUs and MECs in the data layer for virtual agent training, then complete task migration, offloading management, and resource scheduling simulations within IoVs. After identifying the final migration and offloading strategy, task instructions and computational resource scheduling are carried out through a 6G satellite-ground network [15]. Inspired by this approach, our study proposes a DRL algorithm that fuses D3QN and DDPG, named QDPG. For discrete state spaces, including RSU-MEC distances and MEC computational capacities, D3QN effectively minimizes the cost of task offloading selection from RSU to fixed MEC. Moreover, DDPG excels in processing continuous actions for moving vehicles, such as selecting RSUs for task migration. Furthermore, considering the burden placed on the control layer Cybertwin by a large number of digital tasks, this study quantifies digital tasks based on their data size and priority levels. Using the Mini Batch K-Means algorithm, an improvement of the K-Means algorithm, tasks are categorized into different mathematical models. This not only aids Cybertwin in rapidly classifying tasks and efficiently allocating computational resources but also reveals that integrating DRL algorithms within the Mini Batch K-Means architecture input yields certain benefits. Thus, our study amalgamates the advantages of three algorithms, optimizing the latency and energy consumption of task offloading and migration, enhancing post-migration task benefits, and improving MEC resource utilization. Moreover, empirical evidence from our experiments demonstrates the feasibility and superiority of the proposed algorithm compared to other solutions.

The contributions of this paper are as follows:

- (1) This study presents a 6G-integrated space-earth Cybertwin architecture for IoVs. The Cybertwin framework centrally manages computational resources and the scheduling of task migration and offloading, involving entities within the network such as satellites, cloud endpoints, RSUs, MEC servers, and vehicles, addressing the issue of underutilized computational resources in the IoVs architecture.
- (2) We developed a quantified task model that categorizes tasks into high, medium, and low priority levels, as well as large, medium, and small data volume categories, making the task model more aligned with the actual IoVs environment. Utilizing the Mini Batch K-Means algorithm, we cluster similar tasks to assist the Cybertwin control layer in task categorization and computational resource allocation, proposing a benefit function to evaluate the efficiency and effectiveness of task completion.

- (3) To address the issues of service degradation caused by high-speed vehicles in the IoV environment and server overload due to computational resource competition, we introduced a task migration and offloading model. The designed model aims to minimize the delay and energy consumption associated with task migration and offloading, thereby enhancing the efficiency of resource utilization of MEC servers and improving the task benefit for task offload migration.
- (4) In this study, we propose the QDPG algorithm to address the problem of uneven distribution of computational resources in IoV. This algorithm integrates principles from DDPG and D3QN to manage the continuous and discrete action domains in IoV task migration and offloading. Each vehicle user is connected to a shared Critic network, while individual vehicles and RSUs maintain their own Actor networks, thereby facilitating strategic optimization of continuous vehicle actions and improving server selection in the task migration and offloading processes. By adopting D3QN strategies, the shared Critic network is enhanced, thereby increasing the overall stability of the algorithm. Furthermore, in our integrated approach, we innovatively incorporate the Mini Batch K-Means algorithm, further strengthening the fusion DRL algorithm, effectively enhancing task processing efficiency and resource allocation optimization, reducing task processing delays, and improving the overall performance and adaptability of the system.
- (5) Experimental results demonstrate that the QDPG approach is effective in handling task migration and offloading. The results highlight its superiority in terms of reduced delay, improved task benefit, and lower energy consumption when compared to alternative solutions such as the D3QN-DDPG, DQN-DDPG algorithm, D3QN algorithm, DQN algorithm, DDPG algorithm, Greedy Algorithm (GA), and Random Offload (RO) algorithm.

The rest of this paper is structured as follows: Section 2 summarizes previous related work. Section 3 details the model of this paper. Section 4 introduces the proposed QDPG algorithm. Section 5 conducts simulation experiments, and Section 6 provides a conclusion and outlook.

2 | RELATED WORK

The advent of 6G has significantly expanded the scope of IoVs, presenting substantial challenges, especially in the application of terahertz communication links [16–18]. The study by Zhang et al. [17] serves as an example, employing the Dinkelbach method, Lagrangian duality, and mean field games to optimize the application issue of terahertz communication links in 6G networks. Other research [19–21] utilized Cybertwin for timely task offloading. Our study has been greatly inspired by these works, integrating Cybertwin with 6G technology into the IoVs. This integration streamlines network interfaces, facilitates localized training at the Cybertwin layer, enhances the IoVs architecture, and reduces the cost of implementing

optimization algorithms in IoVs, while offering higher IoVs network transmission capabilities and lower task migration offloading processing delays. Considering the instability of 6G terahertz communication, we only employed satellite broadcasting for notifications, effectively minimizing the delay for task notifications and feedback to a negligible level.

The efficacy of Mobile Edge Computing (MEC) technology in task offloading within the IoVs has been well established [22–25], with traditional intelligent algorithms predominantly used for optimizing latency and resources. For instance, research [23] utilized constrained random and heuristic greedy offloading to maximize resources. However, the rapid and cost-free deployment of these conventional algorithms is often impractical within the IoV context. Moreover, traditional heuristic algorithms struggle to address complex, dynamic IoVs issues. In contrast, Deep Reinforcement Learning (DRL) algorithms integrated into the Cybertwin layer offer a model-independent, cost-effective, and efficient deployment advantage for future IoVs scenarios [26, 27]. Notably, [28] employed a DRL algorithm for optimal offloading scheduling, while [29] and [30] applied Dueling DQN combined with Double DQN and pure DDPG, respectively, to enhance task completion, offloading delay, and resource scheduling. This inspired the idea of a fusion algorithm approach. So we consider integrating DRL algorithms to solve the optimal strategy for IoVs task migration and offloading, acknowledging the key issue highlighted in the first section: MEC servers will inevitably become overloaded due to resource competition, and realizing the high cost of directly dynamically relocating MEC servers, a task migration strategy is essential. With the advent of the 6G IoVs era, MEC servers continue to be the primary computing resource within the IoVs network architecture. Hence, optimizing MEC resources through task migration is Critical. Research [31] combined EXP3 and Lyapunov optimization to forecast task migration and minimize migration costs. [32] implemented virtual machine task migration in the cloud to reduce migration costs and optimize server load but did not fully consider the mobility of vehicle users. [33] proposed an incomplete migration strategy, inevitably leading to poor service quality. We are seeking more effective migration and offloading strategies and aim to address these challenges through DRL algorithms. Therefore, we propose an RSU-MEC separation model, dividing the offloading process into migrating to a suitable RSU and then judiciously offloading to the most appropriate MEC server.

We recognize this as a complex action process encompassing both discrete and continuous action domains. This process involves a large number of digital tasks, presenting significant challenges to the Cybertwin control layer. In the dynamic and real-time IoVs, some tasks are urgent, and failure to process them promptly could result in substantial losses. Thus, a smart, orderly, and efficient solution is necessary. Consequently, we apply a vehicle-RSU-MEC separation model within this architecture and innovatively apply the hybrid QDPG algorithm to this IoVs structure. The QDPG algorithm combines the advantages of D3QN in handling discrete actions and DDPG in managing continuous actions, integrating the classification

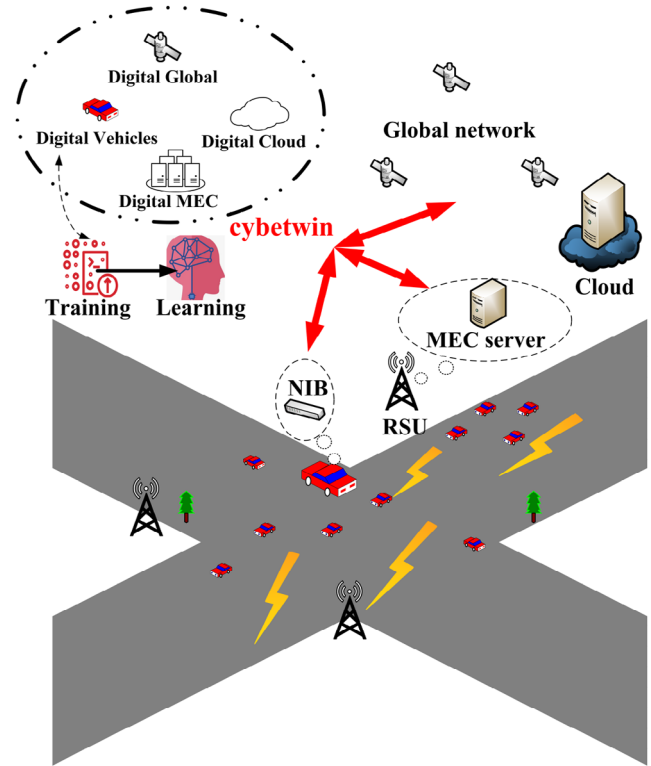


FIGURE 3 System model diagram.

functionality of Mini Batch K-Means. Through Mini Batch K-Means, tasks are classified into high, medium, and low priorities for orderly processing of real-time dynamic tasks and also categorized by size—large, medium, and small. The slicing of large data volume tasks allows for rapid processing and completion of migration and offloading, thereby alleviating the burden on the Cybertwin layer from processing a multitude of tasks and enabling swift allocation of computational resources. Based on this, we construct a task completion score function to facilitate agent learning and iteration, ultimately optimizing the latency and energy consumption of task migration and offloading. It enhances the completion rate of digital application tasks within the vehicular network, increases task benefits, improves the resource utilization rate of MEC servers, and ultimately achieves load balancing among MEC servers.

3 | SYSTEM MODEL

This paper considers a multi-MEC server Internet of Vehicles architecture based on 6G network and Cybertwin technology, as shown in Figure 3. The digital twin Cybertwin technology integrates by creating virtual replicas of physical entities, slicing the real network into: control layer, data layer, and user layer, thereby forming a virtual IoVs.

The data layer is jointly formed by RSUs, MEC servers, and cloud services. We assume that there are S RSUs in the system model, denoted as S_a , where $a \in \{1, 2, \dots, S\}$, corresponding to G MEC servers, G_b , where $b \in \{1, 2, \dots, G\}$.

The user layer in the model is formed by all the mobile vehicles equipped with NIB technology. Vehicle users continuously generate digital tasks during their movement, which are observed and received by the data layer. The data layer communicates data information with the control layer through the 6G satellite network. Specifically, for the user layer, A vehicles equipped with NIBs, A_g , where $g \in \{1, 2, \dots, A\}$, as shown in Figure 3. The k th task in vehicle A_g , $T_{g,k}$, has a demand quantitatively represented as the set $T_{g,k} = \{D_{g,k}, T_{g,k}^m, Z_{g,k}, C_{g,k}, Task_{g,k}\}$, ($k \in [1, K]$), where $D_{g,k}$ represents the data size of $T_{g,k}$, $T_{g,k}^m$ represents the maximum tolerable delay threshold of $T_{g,k}$, $Z_{g,k}$ represents the task action, including local offloading, migration to RSUs, offloading to MECs, $C_{g,k}$ represents the CPU cycles required by the server to execute the task. $Task_{g,k}$ represents the task priority.

In our model, vehicles equipped with NIB technology autonomously manage tasks, communicate with the cloud, and interact with RSUs. We have introduced a Cybertwin layer as the control layer within the IoV architecture, acting as the brain of the vast vehicular network infrastructure, with the function of creating a digital twin of the Internet of Vehicles. This includes compiling a range of data, such as vehicle status, MEC resources, cloud computing resources, and the task demands $T_{g,k}$ of A_g . The generated digital information is sent to a DRL (Deep Reinforcement Learning) agent within the Cybertwin layer. After training, the Cybertwin's agent issues task processing instructions to vehicles, the cloud, and RSUs via the satellite network. It then monitors the status of task completion and relays the results back to the NIB of each vehicle user via the satellite network.

Wireless communication between vehicles and RSUs, as well as between RSUs and MEC servers, has been effectively organized through the IEEE 802.11p standard, effectively establishing links between vehicles and RSUs, as well as between RSUs and MEC servers. In the proposed framework, within the vehicular network equipped with NIBs, the temporal sequence of task migration and offloading is segmented into multiple independent epochs, each denoted as ΔT . By concretizing each task generation slot, we can better understand the sequential differences that exist in the migration and offloading of different tasks. In this system, task storage and migration tasks are solely managed by the RSU, whereas task offloading is delegated to cloud servers, edge servers, and, when necessary, can also be processed by the vehicles themselves. Moreover, the main symbols used in our study are summarized in Table 1.

The vehicle appears in the two-dimensional experimental area:

$$P = \{(x, y) \mid 0 \leq y \leq Y, 0 \leq x \leq X\}. \quad (1)$$

The location information of the vehicle A_g is:

$$PA_g = (Px_g, Py_g). \quad (2)$$

The coverage radius R of the RSU is 400 m and the service node RSU or MEC locations within the vehicle network are:

$$PH_{a_i/b_j} = (Px_{a_i/b_j}, Py_{a_i/b_j}). \quad (3)$$

TABLE 1 Summary of key symbols.

Symbol	Description
S_a	The a th RSU
G_b	The b th MEC
A_g	The g th NIB vehicle
$T_{g,k}$	g th NIB vehicle's k task
$T_{g,k}^m$	Maximum tolerated delay for $T_{g,k}$
$Z_{g,k}$	$T_{g,k}$'s action
$C_{g,k}$	The CPU cycles required by the server to execute the $T_{g,k}$
$Task_{g,k}$	The priority of $T_{g,k}$
$D_{g,k}$	The data size of $T_{g,k}$
$J_{g/b}^a$	The transmission rate between two nodes (g to a or a to b)
$q_{g/b}^a$	The channel gain between two nodes (g to a or a to b)
$d_{g/b}^a$	The distance between two nodes (g to a or a to b)
$B_{T_{g,k}}$	The benefit of $T_{g,k}$
$e_{g,k}^{local}$	The power consumption factor for local offloading
$T_{g,k}^{trans}$	Transmission delay of $T_{g,k}$
$T_{travel}^{RSU_a}$	The travel time in RSU_a of NIB vehicle

3.1 | Communication model

Given the total bandwidth B , Orthogonal Frequency Division Multiple Access (OFDMA) ensures orthogonal spectra for each RSU (Road Side Unit), minimizing interference between RSUs. In this model, all communication nodes utilize wireless communication, with the total bandwidth for communication between NIB vehicles and RSUs being denoted as B_R , and the total bandwidth for communication between RSUs and MEC servers being denoted as B_M . There are N channels available for migrating digital tasks generated by NIBs to MEC servers, represented by n , where $n \in [1, N]$; and S channels for migrating digital tasks generated by NIB vehicles to RSU servers, represented by s , where $s \in [1, S]$. Shannon's theorem helps determine the transmission rate J_g^a and J_b^a for communication nodes in the IoV in two scenarios: J_g^a —interaction between RSU_a and NIB vehicles. J_b^a —interaction between RSU_a and MEC servers G_b .

$$J_g^a = \frac{B_R \times \log_2 \left(1 + \frac{q_{g/b}^a \times P_g^a}{\delta + X_{a,g}} \right)}{S}, \quad (4)$$

$$J_b^a = \frac{B_M \times \log_2 \left(1 + \frac{q_{g/b}^a \times P_b^a}{\delta + X_{a,b}} \right)}{N}, \quad (5)$$

where δ denotes the noise power, P_g^a and P_b^a represent the power coefficients of the two communication models, respectively. $q_{g/b}^a$ denotes the channel gain function between the two nodes:

$$q_{g/b}^a = O \times \left(\frac{3 \times 10^8}{4\pi \times c \times d_{g/b}^a} \right)^L, \quad (6)$$

$$d_{g/b}^a = \sqrt{\left(Px_g - Px_{a_i/b_j}\right)^2 + \left(Py_g - Py_{a_i/b_j}\right)^2}, \quad (7)$$

where O is the antenna gain, c is the carrier frequency, $d_{g/b}^a$ is the distance between two communication nodes expressed in terms of Euclidean distance, and L is the path loss factor. $X_{a,g}$ and $X_{a,b}$, respectively, represent the interference parameters between vehicle task demands within the same channel in two communication models, which are:

$$X_{a,g} = \sum_{g=1, g \neq a}^A \sum_{k=1}^K b_{g,k;a,b} P_g^a q_{g/b}^a, \quad (8)$$

$$X_{a,b} = \sum_{g=1, g \neq a}^A \sum_{k=1}^K m_{g,k;a,b} P_b^a q_{g/b}^a, \quad (9)$$

where $b_{g,k;a,b}, m_{g,k;a,b} \in \{0,1\}$; when $T_{g,k}$, the k -th task of the g -th vehicle, and $T_{a,b}$, the b -th task of the a -th vehicle, are in the same channel, $b_{g,k;a,b}$ and $m_{g,k;a,b}$ are 1, otherwise, they are 0.

3.2 | Priority model

3.2.1 | Prioritisation and classification

Within each time slot ΔT , Cybertwin observes the tasks generated by the users of the NIB vehicles in the IoV, which are classified by Algorithm 1 based on task attributes:

(1) Tasks' data can be divided into large data size (TDL), normal data size (TDN), and small data size (TDS) based on the volume of the data. The TDL tasks can be decomposed into several TDN or TDS tasks for local, MEC, and cloud parallel processing. (2) In IoV, tasks are classified based on execution demands. High-Priority Tasks (HPTs) require strict timeframes and become void if failed. Low-Priority Tasks (LPTs) tolerate latency, losing value only if not completed. Normal Priority Tasks (NPTs), while less strict than HPTs, have time sensitivity—execution duration inversely affecting their value. (3) In evaluating the quality of service in the Internet of Vehicles, network centrality is crucial. Dense task demands in city centers often lead to challenges such as reduced throughput and bandwidth, impacting network centrality. We propose parameters ζ, η , ($\zeta < 1$), ($\eta > 0$), ($\zeta + \eta = 1$), to gauge task performance under unfavorable network conditions. Ideally, ($\zeta > \eta$) for optimal network centrality. When ($\eta > \zeta$), it implies a challenging network environment which may reduce bandwidth and task transmission speed. This means that the interaction difficulty between vehicles and RSU nodes, MEC nodes is increased, increasing latency and energy consumption.

In the real-world vehicular network, we note the prevalence of high-priority tasks (HPTs), such as accident warnings and continuous real-time navigation, which fall under the TDS task category. These are executed through local offloading. In contrast, the cloud is utilized for offloading TDL tasks and low-priority tasks (LPTs), with lenient real-time requirements and large data volumes. To maximize benefit, TDL-sized normal

ALGORITHM 1 Mini batch K-means for task classification.

```

1: Input:  $D_{g,k}, Task_{g,k}, ITER_{max}$ 
2: Output:  $(T_{1,1}, T_{2,1}, \dots), (T_{1,j}, \dots, T_{g,k}), \dots$ 
3: Task feature extraction, preprocessing data. Initialize the initial centroids of  $m$  clusters of  $M$  samples
4: for  $i = 1$  to  $A$  do
5:   for  $j = 1$  to  $K$  do
6:      $t = 0$ 
7:     while  $t < ITER_{max}$  do
8:       Randomly select  $batch\_size$  small batch task sample  $B$ 
9:       for Sample  $x \in B$  do
10:        Use Equation (12) to calculate the distance from  $x$  to each cluster centre  $C_w$ , and assign it to the closest cluster centre  $C_i$ 
11:       end for
12:       for Each cluster  $i$  to  $m$  do
13:        Update the cluster centres of all samples according to Equation (13)
14:       end for
15:       Calculate the SSE for all clusters
16:       if iteration =  $ITER_{max}$  or  $SSE_i = SSE$  then
17:         End
18:       end if
19:     end while
20:   end for
21: end for

```

priority tasks (NPTs) are segmented into TDS-sized and TDN-sized tasks. Following Cybertwin layer optimization, these tasks are offloaded locally, or MEC, or in the cloud according to their action commands.

We employ the Mini Batch K-Means algorithm to cluster similar tasks, enhancing task migration and offloading benefits, and accelerating processing within the Cybertwin layer. This optimized iteration of the K-Means algorithm uses smaller data subsets for faster convergence times, a crucial aspect for time-sensitive IoV tasks.

- (1) Feature extraction, defines numerical features for each task, including the data size and priority of the task. It is assumed that each task $T_{g,k}$ has a data size $D_{g,k}$ and a priority $Task_{g,k}$. $D_{g,k}$ directly uses the numerical range of the data size. $Task_{g,k}$ maps to numerical values (e.g. high, normal, and low correspond to 2, 1, and 0.5, respectively).
- (2) Data preprocessing, which normalises the feature data, ensures that different features have the same weight when calculating the distance. The normalised features are denoted as $N_{D_{g,k}}$ and $N_{Task_{g,k}}$.

$$N_{D_{g,k}} = \frac{D_{g,k} - \min(D)}{\max(D) - \min(D)}, \quad (10)$$

$$N_{Task_{g,k}} = \frac{Task_{g,k} - \min(Task)}{\max(Task) - \min(Task)}. \quad (11)$$

- (3) Initialize M sample tasks as the initial centroids of m clusters ($C_1, C_2, \dots, C_m$). Each centre is a feature vector containing the normalised data size and priority.
- (4) In the t -th iteration, randomly extract some data from the dataset to form a small batch of task samples $\{T_{1,1}, T_{2,1}, \dots, T_{g,k}^{\text{batch_size}}\}$. We assume that some centre is C_w . Employ Equation (12) to determine the distance to each centroid, and allocate the sample set to the class represented by the centroid nearest to it.

$$\text{dist}(T_{g,k}, C_w) = \sqrt{\left(N_{D_{g,k}} - N_{C_w}\right)^2 + \left(N_{T_{g,k}} - N_{C_w}\right)^2}, \quad (12)$$

where $\text{dist}(T_{g,k}, C_w)$ represents the distance between the cluster center C_w and the task sample $T_{g,k}$ belonging to that cluster.

- (5) Update the centre of each cluster to be the average eigenvalue of all tasks within that cluster using Equation (13).

$$C_w^{\text{new}} = \frac{1}{|C_w|} \sum_{T_{g,k} \in C_w} \left(N_{D_{g,k}}, N_{T_{g,k}}\right), \quad (13)$$

where specifically, the fraction $\frac{1}{|C_w|}$ in the formula denotes the operation of averaging, where $|C_w|$ is the number of task samples in cluster C_w . The summation symbol $\sum_{T_{g,k} \in C_w}$ that immediately follows denotes the traversal of all the task samples belonging to the cluster by adding up the normalised data size and the normalised priority of each sample. This process actually computes the average eigenvalue of all task samples in the cluster, which results in the location of the new cluster centre C_w^{new} .

- (6) Cluster quality was assessed using the sum of squared intra-cluster errors (SSE), and iterations were stopped if the SSE no longer changed significantly or if the maximum number of iterations was reached.

$$\text{SSE} = \sum_{g=1}^A \sum_{k=1}^K \text{dist}(C_w, T_{g,k})^2. \quad (14)$$

- (7) When SSE converges or reaches the upper limit of the number of iterations, the algorithm ends and outputs the final clustering results, otherwise repeat steps (3)–(6).

3.2.2 | Task benefits

The task benefits in IoVs are closely related to the delay threshold. Among them, the HPT tasks have the strictest requirements. If the threshold is exceeded, the task is considered a failure, loses its value, and a negative value is given to the

DRL agent as a penalty. The benefit function is:

$$B_{T_{g,k}}^{\text{HPT}} = \begin{cases} \frac{(\eta \times \log(1 + T_{g,k}^m - t_{g,k}^{\text{total}}))}{K_{IDS}^{\zeta}}, & \text{if } t_{g,k}^{\text{total}} \leq T_{g,k}^m, \\ -F(\text{HPT}), & \text{if } t_{g,k}^{\text{total}} > T_{g,k}^m, \end{cases} \quad (15)$$

where $F(\text{HPT})$ is a penalty value to constrain the agent, $t_{g,k}^{\text{total}}$ represents the delay required for task migration, offloading action, and completion, and K_{IDS} is the number of HPTs. Considering NPTs, which include TDS, TDN, and TDL tasks, is key to IoV task benefits due to their number. For TDL tasks, we aim for decomposition and parallel migration offloading, fully utilizing MEC server resources for delay minimization. Although the delay threshold for NPT tasks is relatively high, it decreases exponentially with delay duration:

$$B_{T_{g,k}}^{\text{NPT}} = \begin{cases} \frac{(\eta \times F(\text{NPT}))}{K_{IDN}^{\zeta}}, & \text{if } t_{g,k}^{\text{total}} \leq T_{g,k}^m, \\ F(\text{NPT})^{-e^{\mu(t_{g,k}^{\text{total}} - T_{g,k}^m)}}, & \text{if } t_{g,k}^{\text{total}} > T_{g,k}^m, \end{cases} \quad (16)$$

where K_{IDN} indicates the number of all tasks of which the priority is NPT. $F(\text{NPT})$ is a positive reward value, which decreases exponentially as described above when a task fails, where μ is a penalty coefficient.

Emulating IoV conditions, TPL-priority tasks often consist of data-intensive tasks (e.g. smart apps, video imagery) requiring extended delays for completion without immediate action needs. These tasks are typically offloaded to the cloud. However, if the network's Criticality deteriorates or sudden disruptions occur, task migration might stop, or offloading may fail. In such cases, the benefit value associated with the task defaults to $F(\text{LPT})$:

$$B_{T_{g,k}}^{\text{LPT}} = \begin{cases} \frac{(\eta \times F(\text{LPT}))}{K_{IDL}^{\zeta}}, & \text{if } t_{g,k}^{\text{total}} \leq T_{g,k}^m, \\ -F(\text{LPT}), & \text{Task failed,} \end{cases} \quad (17)$$

where K_{IDL} refers to the number of TDL tasks with a priority of LPT. Finally the overall task benefits are given:

$$B_{T_{g,k}}^{\text{Total}} = \sum_{g=1}^A \sum_{k=1}^K \left(\mathbb{I}(\text{Task}_{\text{HPT}}) B_{T_{g,k}}^{\text{HPT}} + \mathbb{I}(\text{Task}_{\text{NPT}}) B_{T_{g,k}}^{\text{NPT}} + \mathbb{I}(\text{Task}_{\text{LPT}}) B_{T_{g,k}}^{\text{LPT}} - E_{g,k} C_{g,k} \right), \quad (18)$$

where $\mathbb{I}(\text{Task}_{\text{HPT}})$ is an indicator function, with a value of 1 means processing tasks with priority HPT and 0 otherwise. $E_{g,k}$ indicates the energy consumption required to complete the $T_{g,k}$.

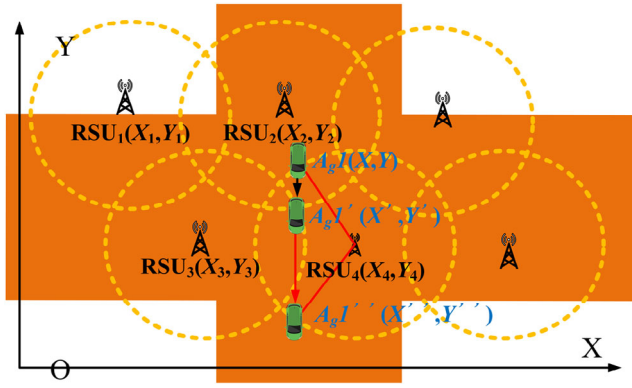


FIGURE 4 Two-dimensional process map for task migration.

3.3 | Migration and offloading model

The Cybertwin layer, in conjunction with the 6G satellite network, leverages DRL for optimizing task migration and offloading, guiding NIB-equipped vehicles and RSUs through THz communication. Task completion information collected at the Cybertwin layer is relayed back to vehicle network users, with the negligible delay enabled by the 6G integrated terrestrial-satellite network. In IoVs, task benefit quality and migration/offloading delay reduction largely depend on MEC servers' robust computing capacity. Hence, optimizing this substantial computing power for offloading is Critical to enhance system efficiency and stability, ultimately benefitting task performance and reducing delays. Consequently, we aim to optimize migration and offloading among vehicles, RSUs, and MECs. Task actions mainly involve local offloading by the vehicle and two-step migration offloading: initial transfer of the task to the RSU for storage, followed by offloading to a specified MEC.

3.3.1 | Migration model

Given the limited coverage of RSUs and the mobility of vehicle users, two conditions must be met for a vehicle task to be migrated to a target RSU. Class (a): the vehicle must be within the RSU's coverage when interacting with it, and Class (b): the transmission delay of the migrated task to the RSU must be shorter than the vehicle's travel time within the RSU's coverage. We present the transmission delay $T_{g,k}^{trans}$ of the vehicle task to the RSU:

$$T_{g,k}^{trans} = D_{g,k} J_g^a. \quad (19)$$

Class (a) is easy to resolve, so next we will provide a mathematical model for Class (b) constraint in two dimensions.

Figure 4 displays this process: Vehicle A_g is driving uniformly from top to bottom at speed V_g . At point (X, Y) , vehicle A_g generates a task offloading and migration requirement. According to Class a, RSU_1 and RSU_3 , which cannot interact with it, will not be considered by the Cybertwin layer. We need to further

calculate the driving times $T_{travel}^{RSU_2}$ and $T_{travel}^{RSU_4}$ of A_g in RSU_2 and RSU_4 , respectively.

Taking $T_{travel}^{RSU_4}$ as an example, the position of A_g is known to us, and the position of RSU_4 is also known. If we want to get $T_{travel}^{RSU_4}$ we must get A_g 's (X'', Y'') , which is the Critical point where it leaves RSU_4 covered, and we can find the distance from the position point of A_g to RSU_4 by first calculating:

$$d_{g1}^{RSU_4} = \sqrt{(X - X_4)^2 + (Y - Y_4)^2}. \quad (20)$$

Bringing in Equations (4) and (6) gives the transmission delay $T_{g,k}^{trans}$ for this task. As long as $T_{travel}^{RSU_4} \geq T_{g,k}^{trans}$ then it can be migrated to RSU_4 :

$$T_{travel}^{RSU_4} = \frac{\sqrt{(X'' - X)^2 + (Y'' - Y)^2}}{V_g}. \quad (21)$$

As A_g travels uniformly downwards, there is:

$$X'' = X. \quad (22)$$

Further, Equation (22) is carried into Equation (21). As A_g travels uniformly downwards, there is:

$$T_{travel}^{RSU_4} = \frac{|Y'' - Y|}{V_g}. \quad (23)$$

The problem is further translated into how to find the position of Y'' , which is obtained from Figure 4, Equation (22), as follows:

$$R = \sqrt{(X_4 - X)^2 + (Y_4 - Y'')^2}, \quad (24)$$

$$Y'' = Y_4 - \sqrt{R^2 - (X_4 - X)^2}. \quad (25)$$

Bringing Equation (25) into Equation (23), it is known that $Y > Y''$.

$$T_{travel}^{RSU_4} = \frac{Y - Y_4 + \sqrt{R^2 - (X_4 - X)^2}}{V_g}. \quad (26)$$

Similarly calculate the time spent from A_g to A_g 's (X', Y') , that is $T_{travel}^{RSU_2}$, where A_g to RSU_2 :

$$d_{g1}^{RSU_2} = \sqrt{(X - X_2)^2 + (Y - Y_2)^2}. \quad (27)$$

Calculate the delay $T_{g,k}^{trans}$ for task migration to RSU_2 using Equation (27).

$$\begin{cases} T_{travel}^{RSU_2} = \frac{\sqrt{(X' - X)^2 + (Y - Y')^2}}{V_g} \\ X' = X \end{cases} \rightarrow T_{travel}^{RSU_2}. \quad (28)$$

Similarly the problem is transformed into finding Y' .

$$\begin{cases} R = \sqrt{(X_2 - X')^2 + (Y_2 - Y')^2} \\ X' = X \end{cases} \rightarrow Y' = Y_2 - \sqrt{R^2 - (X_2 - X)^2}. \quad (29)$$

$T_{travel}^{RSU_2}$ can be found by bringing (29) into (28):

$$T_{travel}^{RSU_2} = \frac{Y - Y_2 + \sqrt{R^2 - (X_2 - X)^2}}{V_g}. \quad (30)$$

The provided model simplifies task migration. In our simulation, vehicles move uniformly in up, down, left, and right directions within a simulated area. If $T_{travel}^{RSU_2} \geq T_{g,k}^{trans}$, and $T_{g,k}$ can migrate the task with RSU_2 , subsequent selections are globally optimized by the Cybertwin layer's DRL algorithm. This solution efficiently addresses the initial problems.

3.3.2 | Offload model

Offloading actions only occur on vehicles equipped with NIB and MEC servers, where the RSU is only responsible for storage and task migration actions.

(1) NIB vehicle local offloading

The local offloading delay $T_{g,k}^{local}$:

$$T_{g,k}^{local} = D_{g,k} F_{NIB}, \quad (31)$$

where F_{NIB} indicates the arithmetic power of the NIB vehicle.

Local offloading of energy consumption $E_{g,k}^{local}$:

$$E_{g,k}^{local} = e_{g,k}^{local} \times T_{g,k}^{local} = \frac{e_{g,k}^{local} \times D_{g,k}}{F_{NIB}}, \quad (32)$$

where $e_{g,k}^{local}$ denotes the power consumption factor for local offloading [34].

(2) MEC offloading

MEC offloading delay comprises vehicle task transmission to RSU ($T_{g,k}^{trans}$), RSU_a task transfer to MEC_b ($T_{g,k}^{a2b}$), and MEC task processing delay. Assuming MEC's computational delay adheres to M/M/1 queuing theory [35], task requests arrive according to ΔT . High-priority tasks are processed first, and tasks of the same priority wait in the MEC server queue according to the order of arrival. Critically, the DRL algorithm selects the MEC server for final offloading based on load balance and remaining computational capacity. If MEC's capacity is insufficient, the Cybertwin instructs the NIB vehicle for local offloading through the satellite network. According to the model case $T_{g,k}^{trans}$ as shown in (19), the analogy yields $T_{g,k}^{a2b}$:

$$T_{g,k}^{a2b} = D_{g,k} J_b^a. \quad (33)$$

According to the M/M/1 queuing model, the arrival tasks of each vehicle user are independent, but the order of task processing differs. Therefore, it's necessary to consider both queuing delay and processing delay based on task arrival rates and server computing power. In the M/M/1 model, this is consolidated into the task processing delay:

$$T_{g,k}^{mec} = 1 / F_{MEC} - \sum_{k=1}^N \omega_{g,k}, \quad (34)$$

where F_{MEC} denotes the initial arithmetic power of the MEC server, $\omega_{g,k}$ denotes the average arrival rate, and $\sum_{k=1}^N \omega_{g,k}$ denotes that there are N tasks arriving at that MEC at this point in the time slot. P_g^a, P_b^a denote the power transmitted in two models, and further calculates the energy cost of MEC offloading $E_{g,k}^{mec}$.

$$E_{g,k}^{mec} = T_{g,k}^{trans} \times P_g^a + T_{g,k}^{a2b} \times P_b^a. \quad (35)$$

Based on the theory of the M/M/1 queuing model, we can observe the resource utilisation RU_a for the a th MEC in this model using Equation (36), similar to

$$RU_a = \sum_{k=1}^N \omega_{g,k} / F_{MEC}. \quad (36)$$

(3) Summary of offloading costs

Total cost of time delay:

$$\begin{aligned} T_{g,k}^{total} = \sum_{g=1}^A \sum_{k=1}^K \left[(\mathbb{I}(\text{Task}_{local})) T_{g,k}^{local} \right. \\ \left. + (\mathbb{I}(\text{Task}_{mec})) (T_{g,k}^{trans} + T_{g,k}^{a2b} + T_{g,k}^{mec}) \right]. \end{aligned} \quad (37)$$

Total cost of energy consumption:

$$E_{g,k}^{total} = \sum_{g=1}^A \sum_{k=1}^K \left[(\mathbb{I}(\text{Task}_{local})) E_{g,k}^{ocal} + (\mathbb{I}(\text{Task}_{mec})) (E_{g,k}^{mec}) \right]. \quad (38)$$

Further, sum the total cost of offloading for subsequent constraint and minimization:

$$U_{g,k} = p T_{g,k}^{total} + q E_{g,k}^{total}, \quad (39)$$

where $(p+q=1)$ represents the weight coefficients of total task delay and total energy consumption; different weight coefficients can guide the DRL algorithm to optimize the preferences of the two objectives. Users can also control the optimization focus by modifying these two parameters.

According to our approach, the more local offloading occurs, the greater the offloading cost. Therefore, our goal is to use MEC offloading as much as possible. However, if all tasks greedily migrate to the nearest MEC for offloading,

it will cause a rapid overload of a particular MEC, trapping the optimization algorithm in a local optimum. We need to consider a formula based on the resource utilization rate of each MEC to assess the load balancing situation of each MEC:

$$MLB = \frac{\sum_{b=1, c \neq b}^G |RU_b - RU_c|}{C_G^2}. \quad (40)$$

Equation (40) is based on the concept of calculating the difference in resource utilisation for each MEC pair and then summing this value with the quotient in the total number of combinations to represent the MEC load balancing (MLB) rate in the system at this point in time. As the number of tasks increases or time slots increase, the resource utilisation (RU) slowly rises, and the smaller the fluctuation in MLB while maintaining a higher value of RU, the more stable or balanced the system is. This suggests that as the number of tasks increases, the intelligent task requests from the NIB vehicles in our model select the MEC servers for final offloading as much as possible, and the MEC servers achieve a higher level of load balancing. Essentially, such a scenario means that we have increased the efficiency of each MEC, optimised task allocation and also ensured a more balanced workload across the network, which means optimising the overall system performance.

4 | PROBLEM CONSTRAINTS AND CODE ANALYSIS

This section will mathematise the task priority-based migration offload problem in the above model into constraints for the DRL algorithm and analyse the QDPG based on improved DDPG and D3QN fusion algorithm proposed in this paper.

4.1 | Problem constraints

According to the content in Section 3, we abstract the problems in the model into constraint conditions, so as to optimize with the DRL algorithm, minimize costs $U_{g,k}$, minimize the fluctuation of MLB , and maximize task benefits $B_{T_{g,k}}^{\text{Total}}$, as follows:

$$\begin{aligned} & \max B_{T_{g,k}}^{\text{Total}}, \min U_{g,k}, \min MLB \\ & \text{s.t.} \\ & C1 : T_{g,k}^{\text{local}} \leq T_{g,k}^m, T_{g,k}^{\text{trans}} + T_{g,k}^{a2b} + T_{g,k}^{\text{mec}} \leq T_{g,k}^m, \\ & C2 : d_g^a \leq R, d_g^r \leq R, C3 : F_{MEC} \geq \sum_{k=1}^N \omega_{g,k}, \\ & C4 : Z_{g,k} \in \{0, 1_{S_a/G_b}\}. \end{aligned} \quad (41)$$

Constraints include: $C1$, task processing delay in both local and migration offloading cannot exceed the task's maximum tolerable delay; $C2$, vehicle must stay within target RSU's coverage before and after migration; $C3$, any MEC's task arrival rate must

not exceed its initial computing power; $C4$, only two task actions exist—local offloading (0) and migration offloading (1), where S_a symbolizes migration to RSU_a , and G_b denotes migration and offloading at MEC_b .

4.2 | QDPG based on improved DDPG and D3QN fusion algorithm

Through analysis of our model, we recognize that task migration and offloading in vehicles equipped with NIB involves both continuous and discrete domains. Vehicles act as intelligent agents executing continuous actions within a state space. The Cybertwin layer must constantly gather data regarding vehicle location, speed, and task status, guiding continuous task migration actions. However, the mode and location selection for task offloading remains discrete. Optimizing such a complex network model poses significant challenges to traditional heuristic algorithms and standalone DRL algorithms. Hence, we propose the QDPG fusion DRL algorithm, merging principles from DDPG and D3QN. This coupled algorithmic approach is an innovative concept in the IoV domain, addressing the complex and common model issues in the discrete and continuous action domains of IoV.

Not content with this alone, noting that digital tasks in the actual IoV are often complex and diverse, we further innovate on the basis of the fusion algorithm by integrating the Mini Batch K-Means clustering algorithm. This approach quantitatively categorizes complex IoV tasks based on two attributes: priority and task size. The DRL algorithm then processes the categorized digital tasks in a second step. The purpose of this is to simplify the complex digital tasks of IoV, while also aligning more closely with the digital task requirements in the future IoV. Moreover, the system can process tasks more effectively, with high-priority tasks receiving timely processing and low-priority tasks scheduled for execution when network and computational resources are optimized. Classifying tasks allows the D3QN-DDPG algorithm to make more informed resource allocation decisions. For example, tasks that require immediate processing can be offloaded to a more powerful or less busy node, or executed immediately at the local NIB. Additionally, by understanding and categorizing the nature of tasks, the system can more effectively balance loads across different IoV nodes (vehicles, RSUs, MEC servers), preventing overload on certain nodes and ensuring a smoother distribution of computational tasks. The algorithm can offload tasks to appropriate nodes in a way that reduces overall latency. The processing speed of high-priority tasks can be accelerated, and large tasks can be split or managed to optimize processing time. Thus, the D3QN-DDPG algorithm can benefit from the structured input provided by Mini Batch K-Means clustering. With more organized data, the learning process can be more efficient, thereby improving the quality of task offloading decisions.

The clustering algorithm is detailed in Section 3, and next, we will elaborate on the specific content of the QDPG algorithm that fuses DDPG and D3QN.

(1) MDP model

Using the Markov process, we construct a model for formula (39). Each deployed NIB vehicle and RSU is an intelligent agent defined in an MDP with three elements: (1) State, (2) Action, (3) Reward. The Cybertwin layer reacts to IoV environment state $S(\Delta T)$ with corresponding action $a(\Delta T)$, evaluates the action's outcome for a reward $r(\Delta T)$, and feeds it back to the IoV environment to achieve the next state $S(\Delta T+1)$.

State: The ΔT state of vehicle A_g includes the position $PA_g(\Delta T)$, the speed $V_g(\Delta T)$, the distance $d_g^a(\Delta T)$ to each RSU, the task transmission rate $J_g^a(\Delta T)$, and the k tasks $T_{g,k}(\Delta T)$ it carries. These tasks include task size $D_{g,k}(\Delta T)$, the required number of CPU cycles $C_{g,k}(\Delta T)$, task priority $Task_{g,k}(\Delta T)$, among other states. In summary, there is a state definition $S_g(\Delta T) = \{PA_g(\Delta T), V_g(\Delta T), T_{g,k}(\Delta T) \dots, S_g(\Delta T)\} \in S_{IoV}$.

Action: The action function of vehicle A_g in the time slot ΔT is $a_g(\Delta T) = \{Task_{g,k}(\Delta T), Z_{g,k}(\Delta T)\}$, where $Task_{g,k}(\Delta T)$ determines the processing order of the task in the action space, and $Z_{g,k}(\Delta T)$ determines the continuous action of the moving vehicle task migrating to the RSU, the discrete action of migration to a fixedly deployed MEC server for offloading from the RSU, or the continuous action of local offloading.

Reward: The reward of this model consists of these parts, with the progression of the time slot, it aims to maximize task benefit, minimize migration offloading cost, and the fluctuation of load balance. The reward function can be expressed as:

$$r(\Delta T) = \sum_{\Delta T=1}^M \left[\mathbb{I}(S) \left(-|MLB(\Delta T) - MLB(\Delta T - 1)| - U_{g,k}(\Delta T) + B_{T_{g,k}}^{Total}(\Delta T) + S_{g,k}(\Delta T) \right) - \mathbb{I}(F) P_{g,k}(\Delta T) \right], \quad (42)$$

where $\mathbb{I}(S)$, $\mathbb{I}(F)$ is an indicator function representing task completion and failure at that time slot. $P_{g,k}(\Delta T)$ is an additional penalty for task failure. Instead, $S_{g,k}(\Delta T)$ is the value of an additional bonus when a task is successful.

(2) QDPG algorithm

In discussing the process of task migration and offloading in the Internet of Vehicles (IoV), our proposed Actor function $a_{g,k}(\Delta T) = \{Task_{g,k}(\Delta T), Z_{g,k}(\Delta T)\}$ encompasses both continuous and discrete action domains. This innovation in the IoV domain quantifies the complex migration and offloading process within vehicle networks into two primary action spaces. This aims to concretize and quantify the complex environment of task migration and offloading in the IoV of the upcoming 6G era. The first step focuses on continuous actions, involving vehicles in motion selecting the appropriate Road Side Unit (RSU), followed by a discrete process determining which tasks should be migrated from RSUs to MEC servers

for offloading. The reason for adopting the hybrid DRL algorithm QDPG is its integration of the strengths of Deep Deterministic Policy Gradient (DDPG) and Dueling Double Deep Q-Networks (D3QN) to address the complexities of continuous and discrete action domains present in IoV task migration and offloading mentioned above. And such fusion can solve the efficiency issues of computational resource allocation in vehicular networks. While the DDPG algorithm excels in dealing with continuous action spaces, its Critic component tends to overestimate in complex and high-dimensional IoV environments, making it challenging for DDPG alone to effectively resolve issues. On the other hand, although the D3QN algorithm shows potential in managing high-dimensional problems, it is limited in exploring continuous action spaces. Therefore, we innovatively combine the advantages of these two algorithms to tackle the complex challenges of IoV task migration and offloading. Our solution digitally simulates these processes within the Cybertwin environment and issues specific offloading action instructions based on simulation outcomes to enhance the quality of mobile services for the next generation of IoV users in the 6G environment. Following this rationale, the action space in our study is divided into two phases: first, the continuous action space $X_{one}^{g,k}(\Delta T) = \{Task_{g,k}(\Delta T), Z_{g,2a}^{g,k}(\Delta T)\}$, where vehicles equipped with NIB technology select target RSUs for task migration based on task priority; subsequently, in the discrete action space $X_{two}^{g,k}(\Delta T) = \{Task_{g,k}(\Delta T), Z_{a2b}^{g,k}(\Delta T)\}$, tasks preliminarily migrated to an RSU are then migrated to an MEC for final offloading as per algorithmic directives.

This innovative design of the action function and action spaces not only addresses the complexities of continuous and discrete action domains but also, by optimizing the allocation of computational resources, provides an efficient and orderly solution for task migration and offloading in the IoV environment. This may signify an important step forward in using DRL algorithms to tackle real-world IoV challenges, especially in supporting the rapidly evolving 6G vehicular network environment.

In this study, we have implemented a fused DRL algorithm, employing the concept of comprehensive training and independent execution for multiple intelligent agents. Each vehicular and RSU intelligent agent is equipped with its own Actor network, while sharing a common Critic network. This architecture enables the Cybertwin layer to make joint decisions during the training and learning process, offering a novel application in enhancing the overall network efficiency and coordination. It allows different agents to make independent decisions while also learning from the feedback of the entire network. D3QN, an improved version of the DQN algorithm, is characterized by its dual Q-networks and Dueling layer. These features address the overestimation issues that could potentially escalate migration-offloading costs or overload MEC servers. The addition of the Dueling layer for value and action advantage estimation provides enhanced stability, accuracy, and

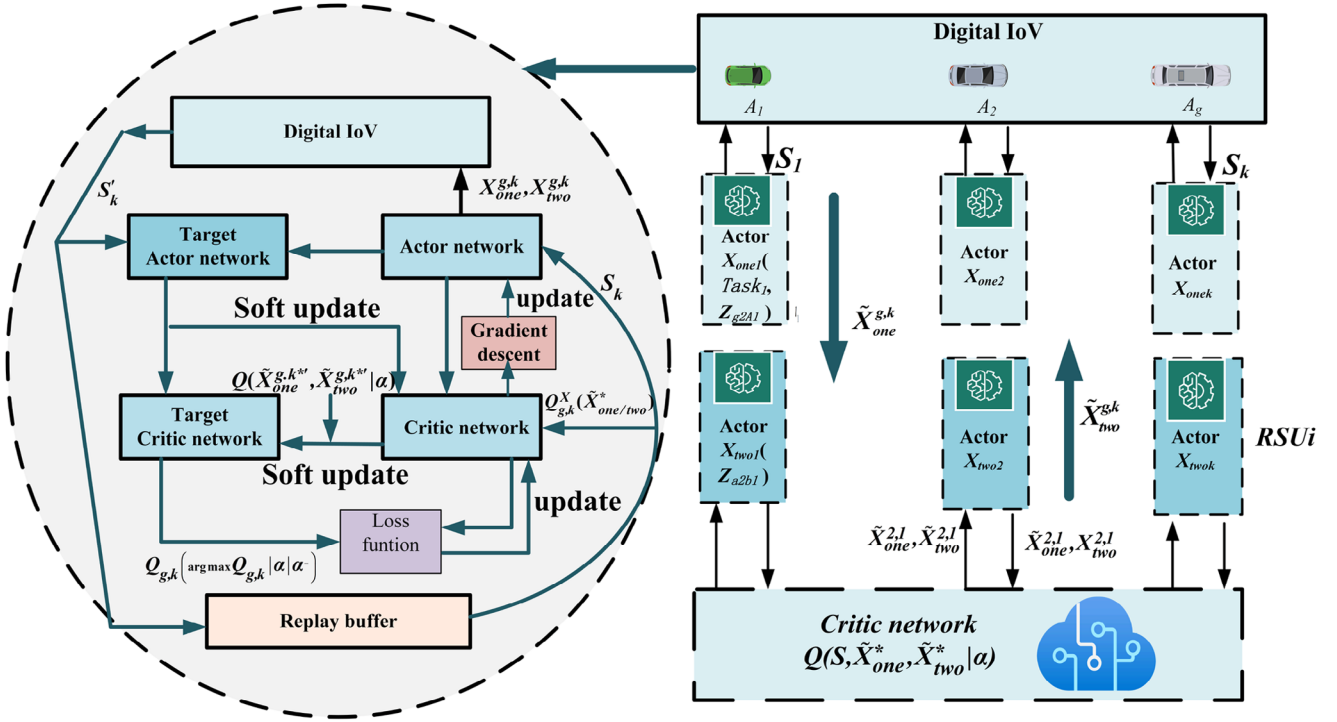


FIGURE 5 QDPG framework diagram.

efficiency. Therefore, we aim to utilize the strengths of D3QN to improve the weaknesses of the traditional Critic network in DDPG, thereby enhancing DDPG's public Critic component, as illustrated in Figure 5. In this setup, the Cybertwin layer digitally represents the IoV, digitizing each vehicle and RSU into individual intelligent agents. Following the principles of D3QN, target networks are deployed separately for the traditional Actor and Critic networks of DDPG, integrated with a Dueling structure, detailed as follows: The vehicle intelligent agent uses its own Actor network function Actor $X_{one}^{g,k} = \{Task_{g,k}, Z_{g2a}^{g,k} | l\}$ to form a continuous action set $\tilde{X}_{one}^{g,k} = (X_{one}^{g,1}, X_{one}^{g,2}, \dots, X_{one}^{g,k})$, and the RSU intelligent agent generates a continuous action set $\tilde{X}_{two}^{g,k} = (X_{two}^{g,1,1}, X_{two}^{g,1,2}, X_{two}^{g,2,1}, \dots, X_{two}^{g,k,1}, \dots, X_{two}^{g,k,k})$ according to the Actor $X_{two}^{g,k}$ information and its own Actor network function Actor $X_{two}^{g,k} = \{Task_{g,k}, Z_{a2b}^{g,k} | l\}$. The shared Critic of the intelligent agents then collects the action sets of all intelligent agents $\tilde{X}_{one}^* = ((\tilde{X}_{one}^{1,1}, \tilde{X}_{one}^{2,1}, \tilde{X}_{one}^{3,2}, \dots), \dots, (\tilde{X}_{one}^{g,k-1}, \tilde{X}_{one}^{g,k}))$, $\tilde{X}_{two}^* = ((\tilde{X}_{two}^{1,2}, \tilde{X}_{two}^{2,1}, \dots), (\dots), (\dots, \tilde{X}_{two}^{g,k}))$ ($\tilde{X}_{one}^*$ represents the sequence of tasks hosted within different RSUs. $\tilde{X}_{two}^*$ represents the upcoming actions of different task sequences within different RSUs under the guidance of QDPG) and the current state S set $S = \{S_1, S_2, \dots, S_k\}$ as the input to the Critic network's action value function $Q(S, \tilde{X}_{one}^*, \tilde{X}_{two}^* | \alpha)$. $Q(S, (\tilde{T}ask', \tilde{U}', \dots) | \alpha)$ as the Critic network's state value function will output the Q value of all NIB vehicle tasks, $Q_{g,k}(S, \tilde{X}_{one}^{g,k}, \tilde{X}_{two}^{g,k} | \alpha)$ will output the Q value of task k

in vehicle g , the final discrete action of task k should be to migrate to MEC_b for offloading or local offloading, which can be obtained through the max $Q_{g,k}$ function, This step actually reflects our incorporation of the core dueling architecture of D3QN into the Critic architecture of DDPG.

$$Z_{a2b}^{g,k} = \arg \max_{X_{one}^{g,k}, X_{two}^{g,k} \in Z_{g,k}} Q_{g,k}(S, \tilde{X}_{one}^{g,k}, \tilde{X}_{two}^{g,k} | \alpha). \quad (43)$$

The implementation process of each NIB vehicle task in the QDPG algorithm is shown in the right part of Figure 5. At the beginning, the digital virtual IoV in Cybertwin performs actions to obtain $r_{g,k}(\Delta T)$, and the next IoV state $S' = \{S'_1, S'_2, \dots, S'_k\}$, and then collects the information in the current virtual IoV into a tuple $(S(\Delta T), \tilde{X}_{one}^*(\Delta T), \tilde{X}_{two}^*(\Delta T), r(\Delta T), S'(\Delta T))$, and puts it in the replay memory for training.

Actor network update: The continuous actions $X_{one}(\Delta T)$ of the NIB vehicle intelligent agents and the continuous actions $X_{two}(\Delta T)$ of the RSU intelligent agents correspond to the continuous migration actions of the intelligent tasks $T_{g,k}$ generated within the IoV, using the following formula to update the Actor network:

$$\begin{aligned} & Q_{g,k}^{X_{one}}(S, (\tilde{X}_{one}^{1,1}, \tilde{X}_{one}^{2,2}, \dots), (\dots), (\dots, \tilde{X}_{one}^{g,k}), X_{two}^{g,k}) \\ &= \max_{X_{one} \in S} Q_{g,k}(S, (\tilde{X}_{one}^{1,1}, \tilde{X}_{one}^{2,2}, \dots), (\dots), (\dots, \tilde{X}_{one}^{g,k}), X_{two}^{g,k}), \end{aligned} \quad (44)$$

$$\begin{aligned} & \mathcal{Q}_{g,k}^{X_{two}} \left(S, \tilde{X}_{one}^{g,k*}, \left(\tilde{X}_{two}^{1,1}, \tilde{X}_{two}^{2,1}, \dots \right), \dots, \left(\dots, \tilde{X}_{two}^{g,k} \right) \right) \\ &= \max_{X_{two} \in G} \mathcal{Q}_{g,k} \left(S, \tilde{X}_{one}^{g,k*}, \left(\tilde{X}_{two}^{1,1}, \tilde{X}_{two}^{2,1}, \dots \right), \dots, \right. \\ & \quad \left. \left(\dots, \tilde{X}_{two}^{g,k} \right) \right). \end{aligned} \quad (45)$$

The gradient is updated from the Actor function to:

$$\nabla \iota = \mathbb{E} \left[\nabla \mathcal{Q}_{g,k}^X \left(S, \left(\tilde{X}_{one}^{1,1}, \tilde{X}_{one}^{2,2}, \dots \right), \left(\dots, \tilde{X}_{one}^{g,k} \right), \right. \right. \\ \left. \left. \left(\tilde{X}_{two}^{1,1}, \tilde{X}_{two}^{2,1}, \dots \right), \left(\dots, \left(\dots, \tilde{X}_{two}^{g,k} \right) \right) \right) \nabla X \right]. \quad (46)$$

Critic network update: The Dueling architecture of the D3QN algorithm separates the state value function V and the action advantage function Λ into two independent entities, subsequently combining them using the average advantage method. This approach is a focal point of our research and a key element in the fusion of DDPG with D3QN. Traditional DDPG Critic networks directly provide Q-values for value and action advantages. However, the introduction of the dueling layer separately estimates the State Value and Action Advantage. This allows the model to more precisely understand the potential value in different states and the relative advantages of executing specific actions over others. Such separation minimizes overestimation and enhances generalization. It ensures that the estimation of state values is not influenced by the size and complexity of the available set of actions. Finally, the average advantage method integrates action and state advantages, facilitating refined value estimations and more effective resource allocation. In summary, this method enhances the efficiency and performance of the algorithm, detailed as follows:

$$\begin{aligned} \mathcal{Q}(S, \tilde{X}_{one}^*, \tilde{X}_{two}^* | \alpha) &= V \left(S, \left(\tilde{I}^{ask'}, \tilde{U}' \dots \right) | \alpha_1, \alpha_2 \right) + \\ & \left(\Lambda(S, \tilde{X}_{one}^*, \tilde{X}_{two}^* | \alpha_1, \alpha_3) - \frac{1}{|\Lambda|} \sum_{j=1}^k \Lambda(S, \tilde{X}_{one}^*, \tilde{X}_{two}^* | \alpha_1, \alpha_3) \right). \end{aligned} \quad (47)$$

Subtracting the mean from our Λ function not only retains the existing order but also enhances the stability and potential improvement of the optimization framework. This means that, even with the adjustment of the mean value, the relative ranking of action advantages remains unchanged. This is a significant improvement over traditional DQN-DDPG fusion algorithms, as it allows the algorithm to effectively differentiate which actions are more advantageous in specific states—a crucial factor in learning how to make optimal choices in varying environmental conditions. Furthermore, regarding the overestimation issue previously mentioned—where traditional reinforcement learning algorithms might excessively value certain actions, leading to biased learning processes—subtracting the mean from each action's advantage in D3QN helps reduce this risk of overestimation, thus increasing the stability and accuracy of the algorithm. By maintaining the relative ranking of action advantages, the algorithm can more precisely evaluate the consequences of different actions in various states. This not only helps avoid errors in strategy selection but also enables the finding of more optimal strategies in complex environments.

In short, this method improves the performance of the algorithm, making it more stable and effective when facing various challenges, ensuring accuracy and consistency in evaluating the relative advantages of different actions.

Besides these improvements, in the current algorithmic structure, there is significant similarity between the values used for action selection and evaluation by the agent network and the target network during the optimization of the Q function. In other words, under the dueling architecture of the algorithm, the risk of overestimation can be reduced, but it's still possible to overvalue certain actions since the same or very similar values are used in selecting and evaluating these actions. This is particularly true in algorithms incorporating DQN, such as DQN-DDPG fusion algorithms, where the target network, used to stabilize the learning process, copies its parameters from the agent network (i.e. the main network). This can lead to the agent network and the target network using overly similar values for action selection and evaluation, leading to overestimation issues. To address this challenge, another key fusion innovation of our research algorithm is proposed: integrating the dual Q-network concept of D3QN into the DDPG network structure. As observed in Figure 5, both the Actor and Critic networks possess target networks, with different Q-value evaluation mechanisms for the target networks compared to the Actor and Critic networks. Thus, even if the parameters of the two networks are similar, the use of different networks for action selection and evaluation further reduces the risk of overestimation. Following this approach, we designate the target network as a separate Q-value evaluation system. The previous formula focuses on how to combine state value and action advantage to calculate the current Q value, used for updating the Critic network, and the following target network update formula involves using dual Q learning in the D3QN framework to update the Q value of the target network. In summary, the former emphasizes the balance between state value and action advantage, while the latter is a strategy for updating Q values in the target network, aimed at reducing overestimation and improving learning stability. The update method is as follows:

$$\begin{aligned} q_{g,k} &= r + \gamma \mathcal{Q}_{g,k} \left(S', \tilde{X}_{one}^{g,k*'}, \tilde{X}_{two}^{g,k*'} \arg \max_{X_{one}, X_{two} \in Z_{g,k}} \right. \\ & \quad \left. \mathcal{Q}_{g,k} \left(S, \tilde{X}_{one}^{g,k*'}, \tilde{X}_{two}^{g,k*'} | \alpha \right) | \alpha^- \right). \end{aligned} \quad (48)$$

In the realm of deep reinforcement learning, particularly in frameworks that employ both target and policy networks, an important observation is the bias in value estimation that can occur during iterative updating processes. This bias may direct policy updates towards suboptimal paths, thereby reinforcing inferior actions. This scenario often leads to a recursive feedback loop where suboptimal actions are not only intensified but also become a persistent feature in subsequent policy updates. Such issues are prevalent in deep reinforcement learning algorithms, especially in dynamic and complex environments, where uncertainties and limitations of the model lead to inaccurate value estimations, making it increasingly difficult to identify

optimal actions as learning progresses. To address these challenges, two measures are considered. The first measure involves increasing the update frequency of the value network, ensuring swift correction of any biases in value estimation. This step is crucial for promptly rectifying errors in value estimation, thereby reducing instability during the learning process. The second measure is to maintain the update frequency of the target network at a level lower than that of the value network. This approach helps to prevent the cumulative error effect caused by frequent updates. In summary, these two measures contribute to enhancing the stability of the learning process, ensuring a smoother progression. Additionally, they help to mitigate the often-cited issue of overestimation in deep reinforcement learning, thereby improving the efficiency of the learning algorithm.

In addition, the Critic part of the QDPG algorithm is updated by minimising a specific loss function (Algorithm 2). This loss function measures the difference between the actual Q value and the predicted Q value.

$$\phi_{g,k}(\alpha) = \mathbb{E} \left[\begin{array}{c} (q_{g,k} - \mathcal{Q}_{g,k}^X(S, (\tilde{X}_{one}^{1,1}, \tilde{X}_{one}^{2,2}), (\dots), \\ (\dots, \tilde{X}_{two}^{g,k}), (\tilde{X}_{two}^{1,1}, \tilde{X}_{two}^{2,1}, \dots), \\ (\dots), (\dots, \tilde{X}_{two}^{g,k})2 \end{array} \right]. \quad (49)$$

In this context, $q_{g,k}$ signifies the target for each iteration of the algorithm, leading to the introduction of the Critic's gradient function. This function plays a crucial role in optimizing the model's parameters, facilitating more accurate estimations of the value function by adjusting the network's weights. Specifically, this formula represents the gradient of the loss function with respect to $\mathcal{Q}_{g,k}$. Updating the Critic network parameters is achieved by computing the gradient of the loss function with respect to $\mathcal{Q}_{g,k}$. By back-propagating this gradient, we can tune the parameters of $\mathcal{Q}_{g,k}$ so that the predicted Q value closer to the actual target Q-value, that is, train the Critic network to more accurately estimate the value of the action. This is a key step in deep reinforcement learning used to optimise strategies and improve the quality of decisions made by intelligences. The implementation details are encapsulated in the following pseudo-code:

$$\begin{aligned} \nabla \phi_{g,k} &= \mathbb{E} \left[\left(q_{g,k} - \mathcal{Q}_{g,k} \left(S, \tilde{X}_{one}^{g,k*}, \tilde{X}_{two}^{g,k*} \right) \right) \right. \\ &\quad \left. \nabla \mathcal{Q}_{g,k} \left(S, \tilde{X}_{one}^{g,k*}, \tilde{X}_{two}^{g,k*} \right) \right]. \end{aligned} \quad (50)$$

(3) Algorithm complexity analysis

We begin by analyzing the algorithmic complexity of the proposed QDPG scheme. In QDPG, initializing Actor and Critic networks is an $O(1)$ operation. The primary loop iterates I times, where for each time slot (iterated T times), and for each vehicle (A vehicles), and for each task (K tasks), the following operations are performed: Action and state collection is $O(1)$ (assumed constant time), action selection is $O(1)$ (constant time), and network updating is $O(U)$, where U represents the complexity of network updating.

ALGORITHM 2 QDPG.

```

1: Receive task processing information from Algorithm 1
2: Initialize Actor network  $X_{one}^{g,k} = \{S, Task_{g,k}, Z_{g,2a}^{g,k}|t\}$  and
    $X_{two}^{g,k} = \{S, Task_{g,k}, Z_{a,2b}^{g,k}|t\}$ , initialize Critic network  $\mathcal{Q}(S, \tilde{X}_{one}^*, \tilde{X}_{two}^*|\alpha)$ 
3: Initialize target Actor network  $X_{one}^{g,k} = \{S, Task_{g,k}, Z_{g,2a}^{g,k}|t^-\}$  and
    $X_{two}^{g,k} = \{S, Task_{g,k}, Z_{a,2b}^{g,k}|t^-\}$ , initialize target Critic network
    $\mathcal{Q}(S, \tilde{X}_{one}^*, \tilde{X}_{two}^*|\alpha^-)$ , where  $t \rightarrow t^-, \alpha \rightarrow \alpha^-$  are the weights
4: Initialize replay memory buffer  $\Theta$ 
5: for episode = 1 to  $ITER^{MAX}$  do
6:   Initialize IoV state  $S$ 
7:   for  $\Delta T_1$  to  $\Delta T_{max}$  do
8:     for vehicle  $A_1$  to  $A_g$  do
9:       for Task 1 to  $k$  do
10:        From  $S(\Delta T)$ , collect continuous actions
         $\tilde{X}_{one}^{g,k} = X_{one}^{g,k}, \tilde{X}_{two}^{g,k} = X_{two}^{g,k}$ 
11:        Selecting discrete actions  $Z_{a,2b}^{g,k}$  with random probability  $\zeta$  and
        Calculating  $B_{T,g,k}^{Total}, U_{g,k}, MLB, Z_{a,2b}^{g,k}$  and reward  $r$ 
12:        Selecting continuous and discrete action
         $\tilde{X}_{one}^{g,k}(\Delta T), \tilde{X}_{two}^{g,k}(\Delta T), Z_{a,2b}^{g,k}(\Delta T)$  and get  $r(\Delta T)$ 
13:        end for
14:      end for
15:    end for
16:    • Update  $(\Delta T)$  action  $X_{one}^{g,k}(\Delta T), Z_{a,2b}^{g,k}(\Delta T)$ ,
      Get  $r(\Delta T)$  and  $S'$  and form tuples
       $(S_{g,k}(\Delta T), \tilde{X}_{one}^{g,k*}(\Delta T), \tilde{X}_{two}^{g,k*}(\Delta T), r_{g,k}(\Delta T), S'_{g,k}(\Delta T))$ . Put it into  $\Theta$ 
17:    Sample a batch of  $i$  tuples from replay memory buffer
18:    for vehicle  $A_1$  to  $A_g$  do
19:      for Task 1 to  $k$  do
20:        From Equation (48) Update QDPG Critic network by
        Equation (48)
21:      end for
22:    end for
23:    if  $\Delta T_i \bmod l$  then
24:      for vehicle  $A_1$  to  $A_g$  do
25:        for Task 1 to  $k$  do
26:          Update QDPG Actor network by (46), and soft update target
          Critic network by  $\tau\alpha + (1 - \tau\alpha^-) \rightarrow \alpha^-, \tau t + (1 - \tau t^-) \rightarrow t^-$ 
27:        end for
28:      end for
29:    end if
30:  end for

```

Thus, the overall updating complexity can be summarized as $O(I \times T \times A \times K \times U)$. Additionally, updating the Actor network is $O(A \times K \times U)$, and updating the Critic network is $O(A \times K \times U)$. Combining these analyses, the total time complexity of QDPG can be estimated as $O(I \times T \times A \times K \times U)$.

For the computational model and the Mini Batch K-Means algorithm used in QDPG, we analyze the complexity

TABLE 2 Complexity analysis of different algorithms.

Algorithm	Complexity
QDPG	5.793×10^7
D3QN-DDPG	5.791×10^7
DQN-DDPG	5.786×10^7
DDPG	4.126×10^7
D3QN	3.526×10^7
DQN	4.781×10^6

of the migration-offloading strategy, which involves calculating the migration and offloading policies for each task (K tasks), potentially including the calculation of transmission rates, delays etc. Let us assume the complexity of each task here is $O(R)$. The complexity of the priority model involves processing each task (K tasks) based on its priority, typically involving simple comparisons and categorizations. Let us assume the complexity of categorizing each task is $O(P)$. Finally, the complexity of Mini Batch K-Means involves initialization $O(m)$, where m is the number of initial centroids.

The main loop iterates $IIER^{\max}$ times, and for each batch (of $batch_size$ size), the complexity of updating centroids is $O(batch_size)$, leading to a total complexity of $O(IIER^{\max} \times batch_size)$.

In summary, when all these parts are combined, the overall time complexity of the algorithm can be approximated as: $O(I \times T \times A \times K \times U) + O(K \times R) + O(K \times P) + O(IIER^{\max} \times batch_size)$. Here, I , T , A , K , and U are the number of iterations, time slots, vehicles, tasks, and the complexity of network updating in the QDPG algorithm, respectively. R and P are the complexities of a single task in the migration-offloading strategy and the priority model. $IIER^{\max}$ and $batch_size$ are the number of iterations and batch size in the Mini Batch K-Means.

Table 2 compares the computational complexities of various DRL algorithms in the proposed model. It is observed that the complexity of our proposed algorithm is slightly higher than that of the D3QN-DDPG fusion algorithm, primarily due to the integration of the clustering algorithm. Compared to DQN-DDPG, both QDPG and D3QN-DDPG exhibit marginally higher complexities. This slight increase can be attributed to the dueling network architecture in D3QN and the integration of the double Q function into the Critic network of DDPG. However, the complexities of these three algorithms are quite close, indicating that such integration does not significantly elevate the overall computational difficulty. Moreover, the Actor-Critic structure of DDPG has a slightly higher complexity than single network algorithms like D3QN. DQN, as a foundational algorithm combining deep learning and reinforcement learning, uses a deep neural network to approximate the Q-value function. Its complexity primarily stems from the network's forward and backward propagation processes, making it the least complex among the compared algorithms.

5 | EXPERIMENTAL SIMULATION

In this section, we will conduct extensive simulation experiments to evaluate the efficiency and accuracy of our proposed model. To ensure consistency and precision, all experiments are performed on a computer with an Intel Core i7 processor, and the experimental simulations are carried out using the Python 3.9 platform.

5.1 | Simulation settings

We propose a new algorithm called QDPG that combines the Actor-Critic structure of DDPG with D3QN, improves the Critic network, and incorporates the Mini Batch K-Means algorithm. In the fusion part of DDPG and D3QN, the Actor network of each vehicle and RSU inputs the migration of tasks from the vehicle to the RSU and the offloading demand from the RSU to the MEC, and the joint Critic network outputs the final offloading action command. The enhanced D3QN-based Critic network processes this data to finalise the MEC selection policy. Notably, the Actor and Critic networks each have two hidden layers of sizes 500 and 450 to manage complex IoV environments. Influenced by D3QN, QDPG's Critic network has a complex input-output layer structure. The input layer handles IoV state information such as the state of NIB vehicles (position, speed, number of tasks), the state of RSUs, and the state of each task (priority, policy). Meanwhile, the output layer provides all vehicle task decisions considering the resource availability of the MEC server within the time slot.

To enhance the predictive accuracy of the QDPG algorithm and reduce the typically optimistic estimates of traditional DQN, our study integrates the principles of D3QN. This integration is a concrete manifestation of the improvement in our research. We have incorporated the Dueling layer from D3QN between the output and hidden layers of the QDPG's Critic network. In our neural network, neurons fulfill different functions: one neuron observes the internal state values of the network, while others estimate the values of potential future actions. Upon integration, the final output layer merges these state values and action advantage predictions to provide a comprehensive MEC server selection scheme. The state value, represented by a single neuron, is a singular number that represents the value of the current state and is independent of any specific action. Using a single neuron for state value estimation reduces the number of parameters in the model, thereby decreasing the risk of overfitting and potentially speeding up the training process.

The Dueling architecture splits the network output into two components: state values and action advantages. This distinction enhances the network's ability to identify the most beneficial actions in different states. As the core concept of the D3QN algorithm, the Dueling architecture facilitates separate learning of state values and action advantages. This enables the network to more accurately understand the overall value of the environment while independently assessing the relative advantage of each potential action. This structure increases the

learning efficiency and the quality of strategies in the D3QN algorithm, thus being incorporated into our proposed QDPG algorithm to enhance its stability and reliability. Our system is thereby enabled to output the optimal migration and offloading scheme while meeting the necessary constraints.

In the algorithm training phase, we meticulously set learning rates for the Actor and Critic networks at 0.0001 and 0.001, respectively. This design ensures a steady improvement in network performance during the learning process, while avoiding overfitting caused by overlearning. We employed a soft target update rate of $\sigma = 0.001$ to ensure the model maintains stability while continually learning and improving. A discount factor of $\gamma = 0.99$ is adopted to encourage the model to appropriately consider future gains during decision-making. Regarding learning strategies, we utilized the ϵ -greedy policy ($\epsilon = 0.9$), a commonly used strategy in reinforcement learning. It not only prompts the model to select the action with the most expected gain in most scenarios, but also to choose sub-optimal actions at a certain probability, thereby ensuring the model's exploratory nature when seeking the optimal strategy. This approach avoids premature convergence to local optimum and neglecting other potential, superior strategies. To further enhance the model's exploratory potential and diversify the model, we introduced Gaussian-distributed $\sim \mathcal{N}(0, 1)$ noise into each action selection. This design causes the model to consider not only current observations and experiences in decision-making, but also a degree of randomness. This means the model is more likely to attempt unconventional strategies, which is crucial for discovering new, effective strategies. Table 3 details some key parameters of the experiment. We hope that our model demonstrates stable and superior performance, both in theoretical analysis and experimental validation.

We consider an experimental simulation area of $1 \text{ km} \times 1 \text{ km}$, where RSUs and MECs are on the same order of magnitude, all set to 5, distributed throughout the experimental area. Vehicles deploying NIBs are randomly generated, conforming to a Poisson distribution, with each vehicle having a different number of task requirements. Tasks are randomly generated in terms of size and priority. The gradual increase in the number of vehicles in the comparison experiments, from 60 to 200 gradually, also implies an increasing number of digital tasks generated. Network Criticality parameters are $\zeta = 0.7$ and $\eta = 0.3$.

The proposed algorithm will be compared with the following algorithms: (1) D3QN: A DQN enhancement, D3QN splits the value function into state value and action advantage components for better action-value estimation. Excelling in high-dimensional state and action spaces, it improves action-value accuracy and task offloading efficiency. It is a popular tool in IoVs task offloading. (2) DDPG: A policy gradient method adept with continuous action spaces, DDPG uses Actor and Critic networks for policy learning and action-value estimation. It is effective in task offloading scenarios and handling complex vehicular network tasks, due to its enhanced policy exploration and optimization capabilities. (3) Hybrid D3QN-DDPG: A hybrid D3QN and DDPG algorithm, exactly as we proposed in the previous section, but without incorporating the Mini Batch K-Means algorithm. (4) Hybrid DQN-DDPG:

TABLE 3 Complexity analysis of different algorithms.

Parameters	Value
Task data size $D_{g,k}$ (3 types)	[(0,10),(10,100),(100,500)] MB
RSU operational coverage	400 m
Priority of task $Task_{g,k}$ (3 types)	[0.5,1,2]
Number of tasks	[100,599]
Number of vehicles	[60,200]
Number of RSUs	5
Number of MECs	5
NIB's computing resources F_{NIB}	[0.1,1] GHz
MEC's computing resources F_{MEC}	[10,20] GHz
Node transmission power P_g^a	0.5w
Node transmission power P_b^a	0.4w
Antenna gain O	3.99
Carrier Frequency c	900 MHz
Path loss coefficient L	2.99
Bandwith B_R	25 MHz
Bandwith B_M	20 MHz
Signal to noise ratio δ	10 dBm
Replay memory buffer Θ	10000
Actor hidden layer	[500,500]
Critic hidden layer	[450,450]
The power consumption fActor for local offloading $e_{g,k}^{local}$	[16,30] J
Speed of vehicles V_g	[10,25] m/s
Training episodes	1000

By integrating DQN and DDPG, Critic network in DDPG benefits from DQN's principles, enhancing its ability to tackle high-dimensional task offloading challenges. This fusion mitigates DDPG's limitations in making discrete offloading action decisions during task offloading [36] (5) DQN. (6) Greedy: Simply choosing the approximate optimal solution for each round without considering long-term returns. (7) Random Offloading (RO).

5.2 | Simulation results and analysis

We compared the convergence of QDPG with DDPG, D3QN, D3QN-DDPG, DQN-DDPG, and DQN algorithms under the same number of vehicles, tasks, RSUs, and MECs. As indicated in Figure 6, all six algorithms converge in 800 episodes, validating their applicability in the vehicle network task migration offloading model.

Within our model, featuring a mix of discrete and continuous actions, the QDPG algorithm's convergence speed, while not the fastest, ultimately resulted in the highest rewards. This underscores the algorithm's applicability in the complex context of vehicular network task migration and offloading. The DDPG algorithm, due to its precise exploration and optimization in

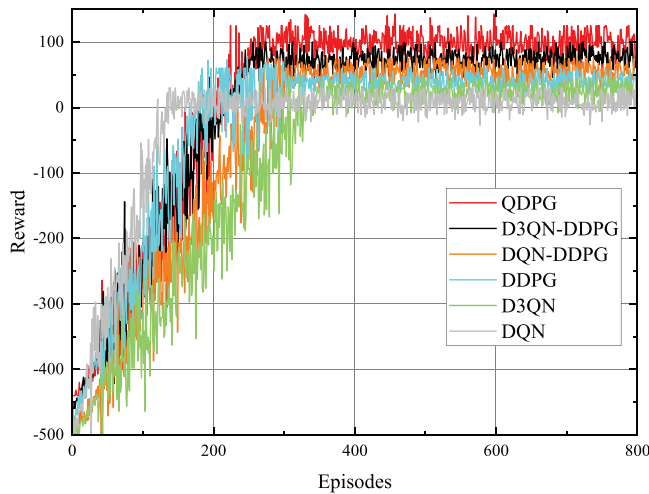


FIGURE 6 Comparison of convergence of algorithms.

continuous action spaces, converges more swiftly. However, its reward acquisition is slightly lower than all hybrid algorithms, constrained by the mixed nature of discrete and continuous actions in the model. Post-convergence, DQN-DDPG outperforms DDPG, benefiting from DQN's capability to solve discrete problems, despite its tendency to overestimate Q-values and a simpler network structure compared to D3QN, which somewhat reduces DQN-DDPG's performance stability and efficacy relative to D3QN-DDPG. Although D3QN excels in policy optimization and high-dimensional decision-making within discrete action domains, its standalone applicability in our model is less than that of the hybrid algorithms. Our proposed QDPG algorithm, while slightly slower in convergence compared to D3QN-DDPG, demonstrates superior performance. This validates the enhanced capability in handling task migration and offloading after integrating the Mini Batch K-Means classification algorithm, leading to more rational resource allocation. The D3QN-DDPG algorithm benefits from the structured input of clustering, exhibiting improved learning and decision-making abilities, thus enhancing the overall system performance. Finally, DQN, despite its fastest convergence, yields the lowest rewards, likely due to its simplistic network structure and granularity issues during the discretization process, leading to less satisfactory performance.

Figure 7 shows that as the number of NIB-equipped vehicle tasks increases, the delay also increases. This indicates that when network congestion occurs or MEC capabilities are maximized, there will be a scarcity of resources and prolonged transmission delays. Interestingly, our proposed QDPG algorithm and the D3QN-DDPG algorithm consistently outperform other algorithms in managing overall task delays. Although DDPG and D3QN excel in continuous and discrete action domains, respectively, they are not effective in handling the complexities of task migration and offloading. Similarly, the performance of DQN-DDPG is superior to pure DDPG and D3QN but slightly inferior to QDPG and D3QN-DDPG. This could be due to the overestimation issues of DQN and the interdependence of Actor and Critic networks in DDPG. Therefore, when the

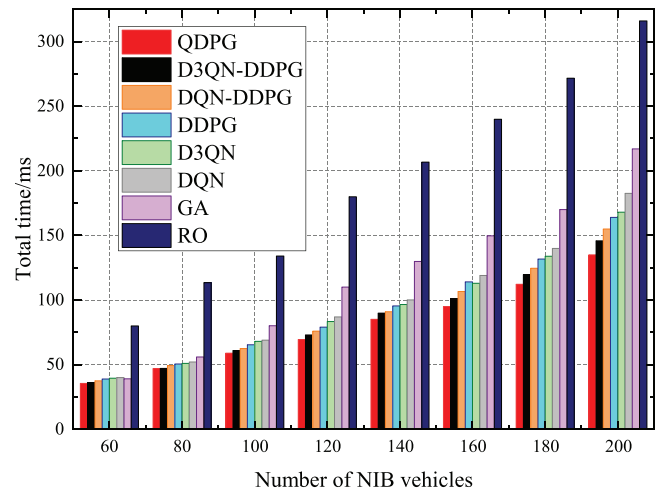


FIGURE 7 Comparison of tasks' latency.

Critic network, improved by DQN, encounters overestimation problems, the performance of the algorithm declines.

Pleasingly, our enhanced QDPG algorithm shows certain advantages over D3QN-DDPG in managing delays in IoV task migration and offloading, and this advantage grows with the increase in the number of vehicles. This indicates that our improved QDPG algorithm, by classifying tasks based on priority, can quickly identify and process urgent or time-sensitive tasks. Tasks requiring real-time responses, such as emergency accident reporting or Critical data transmission, can be prioritized, significantly reducing the response time for these Critical tasks. Moreover, by effectively managing different types of tasks, it is possible to reduce network congestion during peak periods. For instance, less urgent tasks could be processed or migrated when network load is lower, thereby keeping the network clear during high-demand periods. Additionally, by effectively classifying tasks, the system can better adapt to these complex and variable scenarios, such as adjusting task processing strategies in city centers or during traffic peak hours to accommodate higher data traffic and complex network environments. Therefore, QDPG demonstrates the best delay performance, and its advantage becomes increasingly significant as the number of tasks grows.

Figure 8 demonstrates that as the number of NIB task-equipped vehicles increases, the total energy consumption also rises. Despite this trend, our proposed QDPG algorithm consistently shows lower energy consumption compared to other algorithms. This metric is closely related to delays; the longer the delay, the higher the energy consumption. If a DRL algorithm performs suboptimally or lacks stability, leading to extended migration distances, or more frequent local offloading operations due to MEC overloading, this will result in higher delays and, consequently, increased energy costs. Furthermore, in comparison to the hybrid D3QN-DDPG algorithm, QDPG not only reduces processing delays but also enhances the energy efficiency of the IoVs task migration and offloading system by integrating the concept of task classification. By allocating tasks to the most appropriate processing nodes, unnecessary

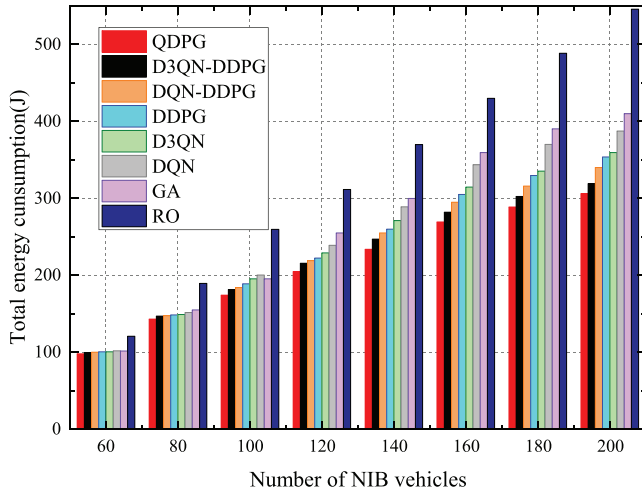


FIGURE 8 Comparison of tasks' energy consumption.

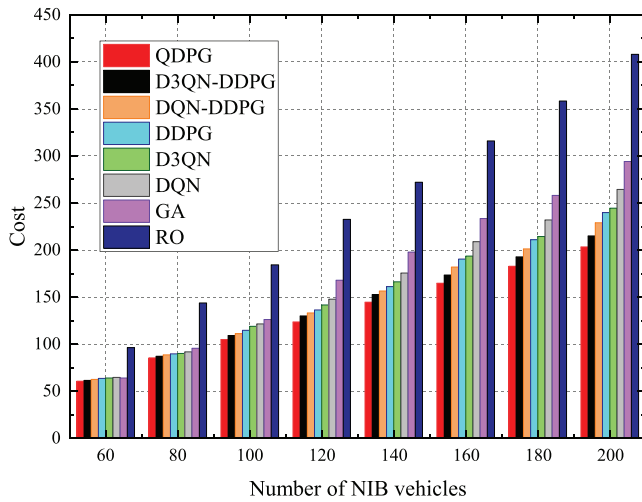


FIGURE 9 Comparison of task migration offload costs ($p = 0.6$, $q = 0.4$).

data transfers and computations can be reduced, thereby lowering overall energy consumption. Therefore, the energy consumption of QDPG is also slightly better than that of the D3QN-DDPG algorithm.

Figure 9 contrasts weighted costs of task migration offloading under different algorithms. We used a weighted scheme emphasizing delay cost. This weighting is predicated on the expectation that in the forthcoming 6G vehicular networks, NIB vehicle users would demonstrate a stronger preference for low-delay service experiences and prompt task responses over energy consumption. The frequency of local offloading actions increases the cost of migration offloading. Our objective is to have tasks choose more efficient MEC offloading, and results show the QDPG algorithm's superior performance.

Figure 10 depicts a simulation based on Equation (36), showcasing Resource Utilization (RU) results of different offloading transfer algorithms. RU indicates the MEC server's resource utilization rate after each task transfer, a key metric here. The experiment includes five RSUs and five MEC servers, the goal is

to attain an RU value that is maximized without surpassing the limit of 1, while also maintaining the resource utilization rates of all five MECs as evenly distributed as possible, representing optimal MEC resource usage.

Figure 10 illustrates the increasing challenges in maintaining balance as the number of vehicular tasks and RU values rise. However, our enhanced QDPG algorithm and the proposed D3QN-DDPG algorithm effectively maintain high RU values and balanced distribution across each MEC. Figure 10a–c,f highlights this exceptional performance, showcasing high resource utilization and robust stability. Figure 10f emphasizes the instability of the RU in the DQN offloading scheme when the number of tasks surges. While the D3QN algorithm uses samples more efficiently than DQN, it struggles with the complexity of continuous action spaces, not as adept as the hybrid DRL algorithms. The DQN-DDPG algorithm, despite utilizing the DQN principle to improve the Critic network, does not overcome the inherent limitations of DQN. Between D3QN and DDPG, D3QN takes the lead as it treats the selection of MEC for offloading as a decision within the discrete action domain. On the other hand, DDPG has experienced overload phenomenon in densely populated task environments. Figures 10d and 10f confirm the superior applicability of D3QN over DQN. GA, favoring local optima without considering long-term returns, underperforms. The rapid increase in the number of tasks from high-density NIB vehicles quickly elevates the utilization rate of certain MEC resources, while the utilization of other MECs remains insufficient. This indicates that the GA algorithm may execute tasks without offloading to MEC, opting for inefficient, high-consumption local offloading actions. Furthermore, our improved QDPG demonstrates higher resource utilization across varying vehicular task numbers compared to D3QN-DDPG, benefiting from the integration of the clustering algorithm. This is attributed to the dynamic resource allocation based on task attributes. Accurate task classification ensures that each task is allocated to the node best suited to its characteristics, thus ensuring the most effective use of resources and minimizing waste, guaranteeing sufficient computational resources and bandwidth for Critical tasks. Among the two high-performing algorithmic approaches, our proposed QDPG shows a greater advantage in optimizing resource utilization.

The MEC load balancing rate closely relates to the Resource Utilization (RU) rates. Large differences in MECs' RU rates per timeslot result in substantial MEC load balancing (MLB) fluctuations, suggesting imbalance in MEC computational resource distribution and a failure in system load balancing. Hence, we extend Equations (34)–(38) for a joint analysis of Figures 11 and 10. We aim to ensure the steady increase of each MEC server's RU value despite fluctuating vehicle task quantities, while keeping differences in RU values across MECs minimal. This approach curtails MLB fluctuations during task quantity surges. The graphs' data points showing significant fluctuations and heightened MLB values warn of potential MEC server overload threats and may suggest under utilization of other MEC server resources.

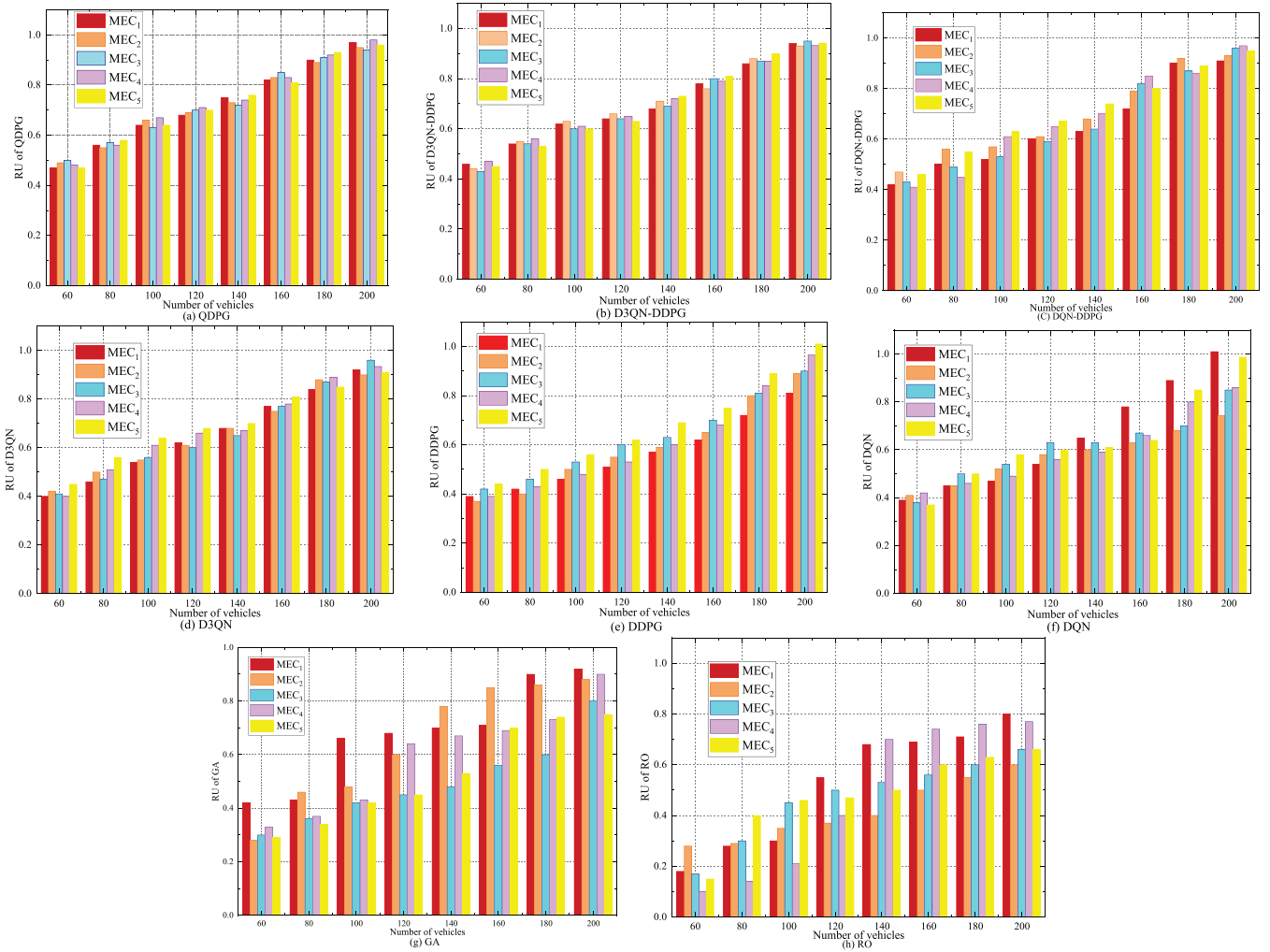


FIGURE 10 Comparison of RU for multiple algorithms.

Figure 11 demonstrates that, compared to other approaches, the QDPG and D3QN-DDPG algorithms consistently achieve lower *MLB* values with less fluctuation. QDPG and D3QN-DDPG consider a range of comprehensive parameters, such as distance from vehicles to nodes, latency, energy consumption generated by task migration and offloading, as well as the computational power of service nodes. This holistic approach minimizes biases in resource allocation, alleviates potential overloading of MEC nodes due to high demand, and ensures that efficient MEC servers are not excluded from task offloading. Even with an increase in the number of vehicles, the *RU* values reported by MECs managed by QDPG and D3QN-DDPG rise but never exceed the Critical threshold of 1, thereby eliminating the risk of MEC server overload. Comparing these two top-tier strategies, QDPG still outperforms, signifying the meaningfulness of our improvements. This is attributed to the incorporation of the clustering algorithm, which further strengthens the allocation of computational resources. It assists the system in more rationally distributing loads across the various nodes in the IOVs, thus preventing certain nodes from

performance degradation due to overload, while also ensuring a smooth distribution of tasks across the network. Therefore, QDPG not only guarantees efficient use of resources but also achieves better load balancing.

Figure 12 presents a comparison of a focus on task migration and offloading completion rates. As task numbers increase, completion rates tend to decline across all algorithms. This is due to the strain placed on MEC servers as task arrival rates increase, leading to a rise in processing latency. This, in turn, can result in total task latency surpassing the maximum latency threshold, leading to task failure. Alternatively, tasks may resort to less efficient local offloading due to MEC servers' capacity constraints, potentially causing the total task delay to exceed the task's threshold. Despite these challenges, the QDPG algorithm shows the least decline in task completion rate, suggesting its effectiveness in handling high-volume vehicle and task scenarios and congested network environments.

Figures 13a and 13b illustrate the variation of task benefits $B_{T_{s,k}}^{Total}$ under the influence of different algorithms and varying numbers of tasks, based on Equation (18). Rewards and

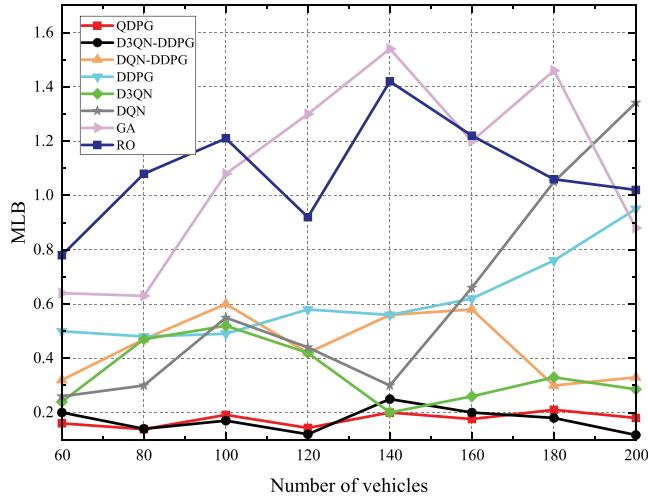


FIGURE 11 Comparison of MECs' load balancing rate MLB for multiple algorithms ($\times 10^{-1}$).

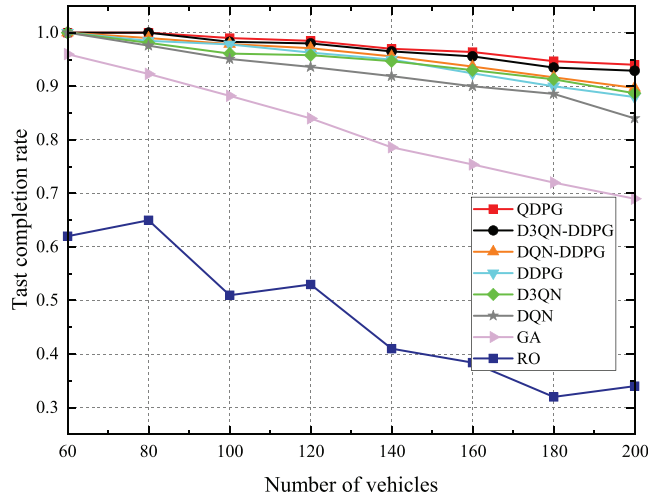


FIGURE 12 Comparison of task completion rates for multiple algorithms.

penalties correspond to the successful completion and failure of tasks, respectively. Therefore, our optimization goal is to achieve higher $B_{T_{g,k}}^{Total}$. As the number of task vehicles increases, the overall value of $B_{T_{g,k}}^{Total}$ under the DRL algorithms also increases, indicating the robust task migration and offloading capability of these algorithms, as depicted in the figures. Excitingly under stable network conditions (when $\zeta = 0.7$ and $\eta = 0.3$), our proposed QDPG algorithm consistently outperforms other similar algorithms in terms of $B_{T_{g,k}}^{Total}$. As showcased in Figure 12b, which demonstrates the changes in $B_{T_{g,k}}^{Total}$ under stringent network conditions. Although the task benefits significantly decrease compared to Figure 12a, the performance of QDPG remains superior to other algorithms. Notably, in highly congested network conditions, the decline in $B_{T_{g,k}}^{Total}$ values observed in the DDPG algorithm is not as pronounced as in the D3QN algorithm, potentially explaining why the

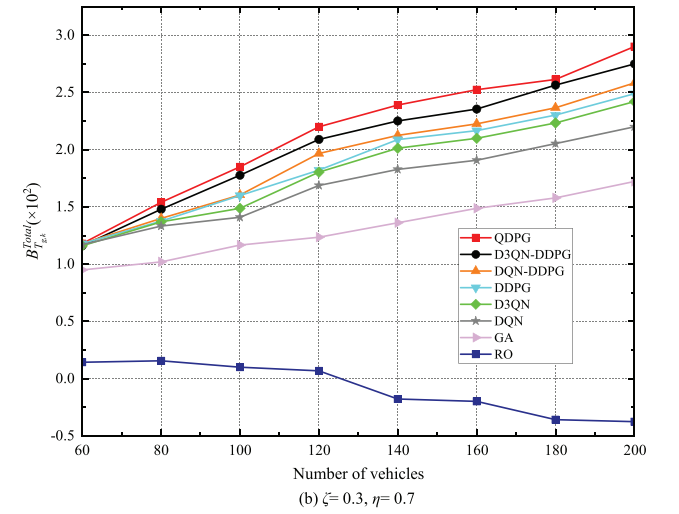
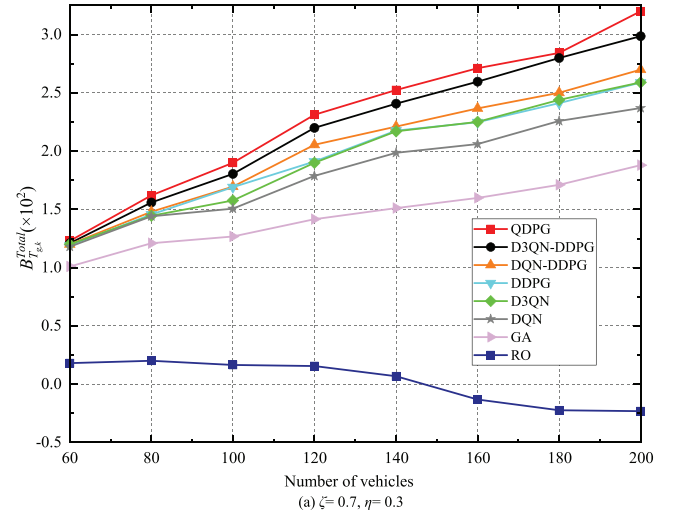


FIGURE 13 Comparison of tasks' benefits $B_{T_{g,k}}^{Total}$ for multiple algorithms.

performance of QDPG, D3QN-DDPG, and DQN-DDPG surpasses that of pure D3QN. The highest task benefit performance of QDPG once again proves that it is a reliable task migration and offloading solution, even in scenarios of severe traffic congestion.

6 | CONCLUSION

In 6G context, we propose an innovative IoV framework based on integrated space-ground systems and the Cybertwin concept. By applying a hybrid QDPG algorithm at the Cyber-twin layer, we address issues of integrating vehicular networks with algorithms and improve MEC server utilization efficiency through a strategy of preliminary task migration before optimal offloading. Experimental results show that the proposed QDPG outperforms other compared methods including D3QN-DDPG, DQN-DDPG, DDPG, D3QN, DQN, GA, and RO, in reducing task latency and energy costs, and improving task completion rates, task benefits and MEC load balancing. We will explore the possibility of MEC server deployment

on drones and the integration of low-Earth-orbit satellite networks into IoV for task transfer offloading in future research. Additionally, we will consider adopting decentralized blockchain technology to address information security concerns in the 6G era.

AUTHOR CONTRIBUTIONS

Rui Wei: Writing—original draft; data curation. **Tuanfa Qin:** Funding acquisition; methodology; project administration; visualization. **Jinbao Huang:** Investigation. **Ying Yang:** Resources; supervision. **Junyu Ren:** Conceptualization; formal analysis. **Lei Yang:** Reviewing and editing.

ACKNOWLEDGEMENTS

This work was supported in part by the Key Research and Development Plan Project of Guangxi, China, under Grant AB23026037, and in part by the Innovation-Driven Development Project of Guangxi, China, under Grant AA20302002-2.

CONFLICT OF INTEREST STATEMENT

The authors declare no conflicts of interest.

DATA AVAILABILITY STATEMENT

Authors elect to not share data.

ORCID

Rui Wei  <https://orcid.org/0009-0001-1495-1304>

Tuanfa Qin  <https://orcid.org/0000-0002-4014-0396>

Jinbao Huang  <https://orcid.org/0009-0006-6129-4171>

Junyu Ren  <https://orcid.org/0000-0003-3701-4900>

REFERENCES

- Stankovic, J.A.: Research directions for the internet of things. *IEEE Internet Things J.* 1(1), 3–9 (2014)
- Kang, J., Yu, R., Huang, X., et al.: Privacy-preserved pseudonym scheme for fog computing supported internet of vehicles. *IEEE Trans. Intell. Transp. Syst.* 19(8), 2627–2637 (2018)
- Samdanis, K., Taleb, T.: The road beyond 5g: A vision and insight of the key technologies. *IEEE Netw.* 34(2), 135–141 (2020)
- Ren, J., Yu, G., Cai, Y., et al.: Latency optimization for resource allocation in mobile-edge computation offloading. *IEEE Trans. Wireless Commun.* 17(8), 5506–5519 (2018)
- Saad, W., Bennis, M., Chen, M.: A vision of 6g wireless systems: Applications, trends, technologies, and open research problems. *IEEE Network* 34(3), 134–142 (2020)
- Tan, L.T., Hu, R.Q.: Mobility-aware edge caching and computing in vehicle networks: A deep reinforcement learning. *IEEE Trans. Veh. Technol.* 67(11), 10190–10203 (2018)
- Shi, J., Du, J., Wang, J., et al.: Priority-aware task offloading in vehicular fog computing based on deep reinforcement learning. *IEEE Trans. Veh. Technol.* 69(12), 16067–16081 (2020)
- Ren, J., Qin, T.: Decentralized blockchain-based and trust-aware task offloading strategy for healthcare IoT. *IEEE Internet Things J.* 11(1), 829–847 (2023)
- Ren, J., Li, J., Liu, H., et al.: Task offloading strategy with emergency handling and blockchain security in sdn-empowered and fog-assisted healthcare iot. *Tsinghua Sci. Technol.* 27(4), 760–776 (2022)
- Ning, Z., Zhang, K., Wang, X., et al.: Intelligent edge computing in internet of vehicles: A joint computation offloading and caching solution. *IEEE Trans. Intell. Transp. Syst.* 22(4), 2212–2225 (2021)
- Rodrigues, T.K., Suto, K., Kato, N.: Edge cloud server deployment with transmission power control through machine learning for 6g internet of things. *IEEE Trans. Emerging Top. Comput.* 9(4), 2099–2108 (2021)
- Zhao, J., Li, Q., Gong, Y., et al.: Computation offloading and resource allocation for cloud assisted mobile edge computing in vehicular networks. *IEEE Trans. Veh. Technol.* 68(8), 7944–7956 (2019)
- Zhang, Q., Yang, L.T., Yan, Z., et al.: An efficient deep learning model to predict cloud workload for industry informatics. *IEEE Trans. Ind. Inf.* 14(7), 3170–3178 (2018)
- Ahrarinnouri, M., Rastegar, M., Seifi, A.R.: Multiagent reinforcement learning for energy management in residential buildings. *IEEE Trans. Ind. Inf.* 17(1), 659–666 (2021)
- Chen, Y., Zhao, F., Chen, X., et al.: Efficient multi-vehicle task offloading for mobile edge computing in 6g networks. *IEEE Trans. Veh. Technol.* 71(5), 4584–4595 (2022)
- Wang, J., Zhu, K., Hossain, E.: Green internet of vehicles (ioV) in the 6g era: Toward sustainable vehicular communications and networking. *IEEE Trans. Green Commun. Networking* 6(1), 391–423 (2022)
- Zhang, Y., Zhang, H., Long, K.: Energy efficient resource allocation in cache based terahertz vehicular networks: A mean-field game approach. *IEEE Trans. Veh. Technol.* 70(6), 5275–5285 (2021)
- Gu, S., Sun, X., Yang, Z., et al.: Energy-aware coded caching strategy design with resource optimization for satellite-uav-vehicle-integrated networks. *IEEE Internet Things J.* 9(8), 5799–5811 (2022)
- Xu, X., Shen, B., Ding, S., et al.: Service offloading with deep q-network for digital twinning-empowered internet of vehicles in edge computing. *IEEE Trans. Ind. Inf.* 18(2), 1414–1423 (2022)
- Chen, Y., Zhao, F., Chen, X., et al.: Efficient multi-vehicle task offloading for mobile edge computing in 6g networks. *IEEE Trans. Veh. Technol.* 71(5), 4584–4595 (2022)
- Hou, W., Wen, H., Song, H., et al.: Multiagent deep reinforcement learning for task offloading and resource allocation in cybertwin-based networks. *IEEE Internet Things J.* 8(22), 16256–16268 (2021)
- Sun, J., Gu, Q., Zheng, T., et al.: Joint optimization of computation offloading and task scheduling in vehicular edge computing networks. *IEEE Access* 8, 10466–10477 (2020)
- Guo, H., Zhang, J., Liu, J.: Fiwi-enhanced vehicular edge computing networks: Collaborative task offloading. *IEEE Veh. Technol. Mag.* 14(1), 45–53 (2019)
- Ni, Z., Chen, H., Li, Z., et al.: MSCET: A multi-scenario offloading schedule for biomedical data processing and analysis in cloud-edge-terminal collaborative vehicular networks. *IEEE/ACM Trans. Comput. Biol. Bioinf.* 20(4), 2376–2386 (2023)
- Zhou, Z., Liao, H., Wang, X., et al.: When vehicular fog computing meets autonomous driving: Computational resource management and task offloading. *IEEE Netw.* 34(6), 70–76 (2020)
- Peng, H., Shen, X.: Deep reinforcement learning based resource management for multi-access edge computing in vehicular networks. *IEEE Trans. Network Sci. Eng.* 7(4), 2416–2428 (2020)
- Manogaran, G., Mumtaz, S., Mavromoustakis, C.X., Pallis, E., Mastorakis, G.: Artificial intelligence and blockchain-assisted offloading approach for data availability maximization in edge nodes. *IEEE Trans. Veh. Technol.* 70(3), 2404–2412 (2021)
- Zhan, W., Luo, C., Wang, J., et al.: Deep-reinforcement-learning-based offloading scheduling for vehicular edge computing. *IEEE Internet Things J.* 7(6), 5449–5465 (2020)
- Tang, M., Wong, V.W.S.: Deep reinforcement learning for task offloading in mobile edge computing systems. *IEEE Trans. Mob. Comput.* 21(6), 1985–1997 (2022)
- Chen, Z., Wang, X.: Decentralized computation offloading for multi-user mobile edge computing: A deep reinforcement learning approach. *EURASIP J. Wirel. Commun. Network.* 2020, 188 (2020)
- Labriji, I., Strinati, E.C., Perraud, E., et al.: Dynamic migration strategy for mobile multi-access edge computing services. In: 2022 IEEE Wireless Communications and Networking Conference (WCNC), pp. 710–715. IEEE, Piscataway (2022)

32. Zhou, A., Wang, S., Ma, X., et al.: Towards service composition aware virtual machine migration approach in the cloud. *IEEE Trans. Serv. Comput.* 13(4), 735–744 (2020)
33. Peng, Q., Xia, Y., Feng, Z., et al.: Mobility-aware and migration-enabled online edge user allocation in mobile edge computing. In: 2019 IEEE International Conference on Web Services (ICWS), pp. 91–98. IEEE, Piscataway (2019)
34. Cui, L., Xu, C., Yang, S., et al.: Joint optimization of energy consumption and latency in mobile edge computing for internet of things. *IEEE Internet Things J.* 6(3), 4791–4803 (2019)
35. Mach, P., Becvar, Z.: Mobile edge computing: A survey on architecture and computation offloading. *IEEE Commun. Surv. Tutor.* 19(3), 1628–1656 (2017)
36. Zhang, J., Du, J., Shen, Y., et al.: Dynamic computation offloading with energy harvesting devices: A hybrid-decision-based deep reinforcement learning approach. *IEEE Internet Things J.* 7(10), 9303–9317 (2020)

ment learning approach. *IEEE Internet Things J.* 7(10), 9303–9317 (2020)

How to cite this article: Wei, R., Qin, T., Huang, J., Yang, Y., Ren, J., Yang, L.: Resource allocation scheduling scheme for task migration and offloading in 6G Cybertwin internet of vehicles based on DRL. *IET Commun.* 18, 1244–1265 (2024).
<https://doi.org/10.1049/cmu2.12826>